



Red Hat OpenShift AI Self-Managed 3.5

Working with OGX

Working with OGX in Red Hat OpenShift AI Self-Managed

Red Hat OpenShift AI Self-Managed 3.5 Working with OGX

Working with OGX in Red Hat OpenShift AI Self-Managed

Legal Notice

Copyright © Red Hat.

Except as otherwise noted below, the text of and illustrations in this documentation are licensed by Red Hat under the Creative Commons Attribution–Share Alike 3.0 Unported license . If you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, the Red Hat logo, JBoss, Hibernate, and RHCE are trademarks or registered trademarks of Red Hat, LLC. or its subsidiaries in the United States and other countries.

Linux[®] is the registered trademark of Linus Torvalds in the United States and other countries.

XFS is a trademark or registered trademark of Hewlett Packard Enterprise Development LP or its subsidiaries in the United States and other countries.

The OpenStack[®] Word Mark and OpenStack logo are trademarks or registered trademarks of the Linux Foundation, used under license.

All other trademarks are the property of their respective owners.

Abstract

As a cluster administrator, you can use the OGX Operator in Red Hat OpenShift AI.

Table of Contents

CHAPTER 1. LLAMA STACK TO OGX MIGRATION	5
1.1. MIGRATING FROM LLAMA STACK TO OGX	7
CHAPTER 2. OVERVIEW OF OGX	10
CHAPTER 3. SECURITY REFERENCE FOR OGX	12
3.1. NETWORK EXPOSURE	12
3.2. TLS GUIDELINES	15
3.2.1. Server-level TLS and FIPS cipher enforcement	15
3.2.2. CA bundle configuration	16
3.3. AUTHENTICATION GUIDELINES	17
3.3.1. Option 1: Direct token validation	17
3.3.2. Option 2: Gateway-delegated authentication	18
3.3.3. Authentication providers	18
3.3.4. UpstreamHeaderAuthProvider trust model	22
3.4. AUTHORIZATION GUIDELINES	23
3.4.1. Route-level authorization	23
3.4.2. Resource-level ABAC	23
3.4.3. Tenant isolation	24
3.4.4. Access policy configuration reference	25
3.5. SECRETS MANAGEMENT	28
3.5.1. Operator secret collection mechanism	29
3.5.2. Provisioning and rotation	30
3.6. NETWORKPOLICY CONFIGURATION	31
3.6.1. Default NetworkPolicy rules	31
3.6.2. Custom network access rules	31
3.7. GUARDRAILS INTEGRATION SECURITY	33
3.7.1. OGX Responses API guardrails	33
3.7.2. RHOAI NeMo guardrails integration	34
3.8. HARDENING RECOMMENDATIONS	34
3.8.1. Container security context	35
3.8.2. Network segmentation	36
3.8.3. Audit logging	37
CHAPTER 4. OGX APIS	39
4.1. SUPPORTED OGX APIS IN OPENSIFT AI	39
4.1.1. File Processors API	39
4.1.2. Datasets_IO API	39
4.1.3. Inference API	39
4.1.4. Tool Runtime API	39
4.1.5. Vector_IO API	40
4.2. OPENAI-COMPATIBLE APIS IN OGX	40
4.2.1. Supported OpenAI-compatible APIs in OpenShift AI	40
4.2.1.1. Models API	41
4.2.1.2. Chat Completions API	41
4.2.1.3. Completions API	42
4.2.1.4. Embeddings API	42
4.2.1.5. Files API	43
4.2.1.6. Vector Stores API	45
4.2.1.7. Vector Store Files API	45
4.2.1.8. Responses API	45
4.2.1.9. Conversations API	47

4.2.2. OpenAI compatibility for RAG APIs in OGX	49
4.3. OGX API PROVIDER SUPPORT	50
4.4. OPENAI-COMPATIBLE FILE CITATION ANNOTATIONS	53
4.4.1. OpenAI-compatible file citation annotations in OGX	53
4.4.2. Viewing file citation annotations in Responses API output	54
4.4.3. File citation annotation reference	56
4.4.3.1. Annotation location	56
4.4.3.2. Supported annotation type	56
4.4.3.3. File citation fields	57
4.4.3.4. Annotation behavior	57
CHAPTER 5. ACTIVATING THE OGX OPERATOR	58
CHAPTER 6. DEPLOYING A OGX SERVER	60
CHAPTER 7. TESTING YOUR VLLM MODEL ENDPOINTS	63
CHAPTER 8. CONFIGURE AWS BEDROCK AS AN OGX INFERENCE PROVIDER	66
CHAPTER 9. DEPLOYING A LLAMA MODEL WITH KSERVE	70
CHAPTER 10. SELECT AND DEPLOY A VECTOR DATABASE	74
10.1. OVERVIEW OF VECTOR DATABASES	74
10.1.1. Overview of Milvus vector databases	75
10.1.2. Overview of pgvector vector databases	76
10.2. DEPLOYING A REMOTE MILVUS VECTOR DATABASE	77
10.3. USING POSTGRESQL IN OGX	81
10.3.1. Understanding OGX metadata storage	81
10.3.1.1. Role of metadata storage in OGX	82
10.3.1.2. PostgreSQL metadata storage backends	82
10.3.2. Deploying a PostgreSQL instance with pgvector	82
10.3.3. Configuring the pgvector remote provider in OGX	86
10.3.4. Resources created when you create a playground	88
10.3.5. Configure a custom PostgreSQL instance for playground RAG	89
10.4. USING QDRANT IN OGX	91
10.4.1. Overview of Qdrant vector databases	92
10.4.2. Deploying a Qdrant vector database	92
10.4.3. Configuring the Qdrant remote provider in OGX	95
10.4.4. Performing vector operations with Qdrant	98
10.4.4.1. Add files to a vector store	99
10.4.4.2. Query a vector store	99
10.4.4.3. Delete a vector store	99
10.5. CONFIGURE EXTERNAL VECTOR STORES FOR THE PLAYGROUND	100
CHAPTER 11. DEPLOY OGX FOR MULTI-TENANCY	103
11.1. OVERVIEW OF MULTI-TENANCY ON OGX	103
11.1.1. Single-server vs Multi-server environments	103
11.1.2. Roles for multi-tenancy environments	103
11.1.3. Operator-enforced isolation	104
11.2. CREATING A SINGLE-SERVER MULTI-TENANT ENVIRONMENT	104
11.3. CREATING A MULTI-SERVER MULTI-TENANT ENVIRONMENT	109
11.4. USING APIS AS A TENANT USER	112
CHAPTER 12. CONFIGURING OGX WITH OAUTH AUTHENTICATION	115
CHAPTER 13. CONFIGURE ATTRIBUTE-BASED ACCESS CONTROL (ABAC) ON YOUR OGX SERVER	120

CHAPTER 14. SELF-SIGNED CERTIFICATES WITH OGX	123
CHAPTER 15. ENABLING HIGH AVAILABILITY AND AUTOSCALING FOR OGX	124

CHAPTER 1. LLAMA STACK TO OGX MIGRATION

Starting in OpenShift AI version 3.5EA1, Llama Stack is being fully renamed to OGX. This creates breaking changes in any applications created with the Llama Stack Operator.

The following charts show the naming changes of the components, environment variables, and field changes.

Table 1.1. Name Mapping

Component	Previous name	New name
API Group	llamastack.io	ogx.io
API Version	v1alpha1	v1beta1
Kind	LlamaStackDistribution	OGXServer
Plural	llamastackdistributions	ogxservers
Short Name	llsd	ogxserver
Container Name	llama-stack	ogx
App Label	app: llama-stack	app: ogx
Managed-by	llama-stack-operator	ogx-operator
Watch Label	llamastack.io/watch: "true"	ogx.io/watch: "true"
Mount Path	/.llama	/.ogx
Leader Election ID	81d5736e.llamastack.io	54e06e98.ogx.io

Table 1.2. Environment Variables

Previous name	New name	Additional details
LLS_PORT	OGX_PORT	Container port for the server
LLS_WORKERS	OGX_WORKERS	Number of uvicorn worker processes
LLAMA_STACK_CONFIG	OGX_CONFIG	Path to the server config file

Table 1.3. Status Field Changes

Old Path	New Path
<code>.status.version.llamaStackServerVersion</code>	<code>.status.version.serverVersion</code>
<code>.status.routeURL</code>	<code>.status.externalURL</code>

The following YAML examples display the changes in specifications. For example: OGXServer CRs, network configurations, and workload configurations.

Workload Configuration

Previous workload configuration (flat on spec):

```
spec:
  replicas: 2
  server:
    distribution:
      name: rh-dev
  containerSpec:
    env:
      - name: MY_VAR
        value: "hello"
  storage:
    size: "20Gi"
```

New workload configuration (grouped under **spec.workload**):

```
spec:
  distribution:
    name: rh-dev
  workload:
    replicas: 2
    storage:
      size: "20Gi"
    overrides:
      env:
        - name: MY_VAR
          value: "hello"
```

Network Configurations

Previous network configurations (**spec.network**):

```
spec:
  network:
    exposeRoute: true
    allowedFrom:
      namespaces: ["my-app"]
      labels: ["team=frontend"]
```

New network configurations(**spec.network**):

```
spec:
```

```

network:
  externalAccess:
    enabled: true
  policy:
    enabled: true
  ingress:
    - from:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: my-app
        - namespaceSelector:
            matchLabels:
              team: frontend
    ports:
      - protocol: TCP
        port: 8321

```

1.1. MIGRATING FROM LLAMA STACK TO OGX

In order to migrate to the newly named **ogx-operator**, you must remove the Llama Stack Operator and create new OGXServer custom resources (CRs).

Prerequisites

- You have the Llama Stack Operator installed on your OpenShift AI cluster.
- You have custom **LlamaStackDistribution** applications.
- You have cluster administrator permissions.
- You have installed the OpenShift CLI (**oc**).

Procedure

1. Remove the Llama Stack Operator from your environment. You can remove the Llama Stack Operator by setting the component spec:

```
$ dsc.spec.components.lls = "Removed"
```

2. Install the new OGX operator by setting the component spec:

```
$ dsc.spec.components.ogx = "Managed"
```

3. Create the **OGXServer** custom resource (CR).

```

apiVersion: ogx.io/v1beta1
kind: OGXServer
metadata:
  name: my-server
spec:
  distribution:
    name: rh-dev
  workload:
    replicas: 1

```

```
storage:
  size: "20Gi"
overrides:
  env:
    - name: OLLAMA_INFERENCE_MODEL
      value: "llama3.2:1b"
    - name: OLLAMA_URL
      value: "http://ollama-server-service.ollama-dist.svc.cluster.local:11434"
```

4. Apply the **OGXServer** CR to the cluster:

```
$ oc apply -f ogxserver.yaml
```

Verification

1. Verify the pod deployment with the following command:

```
# Check the new CRD is registered
$ oc get crd ogxservers.ogx.io

# List OGXServer resources
$ oc get ogxserver

# Check conditions for adoption status
$ oc get ogxserver my-server -o jsonpath='{.status.conditions}'

# Verify the server is ready
$ oc get ogxserver my-server -o jsonpath='{.status.phase}'
```

2. You can then clean up the legacy resources when the new OGXServer is verified.

- a. Remove the **LlamaStackDistribution** CR resources:

```
$ oc delete llamastackdistribution <old-llsd-name> -n <namespace>
```

3. (Optional) Adopting existing PVC.

- a. To preserve existing data by adopting the PVC from the old **LlamaStackDistribution**, set the **annotations** parameter similar to the following:

```
metadata:
  annotations:
    ogx.io/adopt-storage: "<old-llsd-name>"
```

The operator strips the old **ownerRef** from the PVC and labels it for discovery. The adopted PVC intentionally has no **ownerReference** to the OGXServer.

4. (Optional) Adopting existing Service and Ingress

- a. To preserve ClusterIP and external endpoints, set the **annotations** similar to the following:

```
metadata:
  annotations:
    ogx.io/adopt-storage: "<old-llsd-name>"
```

■ `ogx.io/adopt-networking: "<old-llsd-name>"`

The operator adopts the orphaned Service + Ingress, replaces Service selectors with new pod labels: **app: ogx**, **app.kubernetes.io/instance: <name>**, and sets **ownerReferences**.

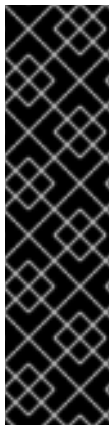
CHAPTER 2. OVERVIEW OF OGX

OGX is a unified AI runtime environment designed to simplify the deployment and management of generative AI workloads on OpenShift AI. In OpenShift, the OGX Operator manages the deployment lifecycle of these components, ensuring scalability, consistency, and integration with OpenShift AI projects. OGX integrates model inference, embedding generation, vector storage, and retrieval services into a single stack that is optimized for retrieval-augmented generation (RAG) and agent-based AI workflows.

OGX concepts

- **OGX Operator** Installs and manages OGX server instances in OpenShift AI, handling lifecycle operations such as deployment, scaling, and updates.
- **The `run.yaml` file** Defines which APIs are enabled and how backend providers are configured for a OGX server. Red Hat ships a default **`run.yaml`** that supports common deployment scenarios. You can provide a custom **`run.yaml`** to enable advanced workflows or integrate additional providers.
- **OGXServer custom resource** Declares the runtime configuration for a OGX server, including model providers, embedding configuration, vector storage, and persistence settings.

OpenShift AI ships with a OGX Distribution that runs the OGX server in a containerized environment. For the OGX Operator version included in this release of OpenShift AI, see [Supported Configurations for 3.x](#).



IMPORTANT

OGX integration is currently available in Red Hat OpenShift AI 3.5 as a Technology Preview feature. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production.

These features provide early access to upcoming product capabilities, enabling customers to test functionality and provide feedback during development.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

OGX includes the following core components:

- **Integration with OpenShift AI** Uses the **OGXServer** custom resource to simplify configuration and deployment of AI workloads.
- **Inference model connections** Acts as a proxy between OGX APIs and model inference servers, such as vLLM deployments.
- **Embedding generation** Generates vector embeddings used for retrieval. In OpenShift AI 3.2, remote embedding models are the recommended and default option for production deployments. Inline embedding models remain available for development and testing scenarios.
- **Vector storage** Stores and indexes embeddings by using supported vector databases, such as Milvus or PostgreSQL with the pgvector extension.

- **Metadata persistence** Stores vector store metadata, file references, and configuration state. In OpenShift AI 3.2, PostgreSQL is the default backend for production-grade deployments.
- **Retrieval workflows** Manages ingestion, chunking, embedding, and similarity search to support RAG workflows.
- **Agentic workflows** Enables agent-based interactions through supported APIs, such as OpenAI-compatible Responses and Chat Completions.

For information about deploying OGX in OpenShift AI, see [Deploying a RAG stack in a project](#) .



NOTE

The OGX Operator is not currently supported on IBM Z platform.

OGX is supported on IBM Power (**ppc64le**) with limited functionality:

- The GenAI playground is supported and available on the IBM Power architecture.
- **milvus-lite** is supported and available as a vector store option on the IBM Power architecture.
- Although **PostgreSQL** with the **pgvector** extension is listed as a supported vector store, it is not currently available on the IBM Power ppc64le architecture.

Additional resources

- [OGX demos repository](#)
- [OGX Kubernetes Operator](#)
- [OGX documentation](#)

CHAPTER 3. SECURITY REFERENCE FOR OGX

The following documentation details the security architecture of OGX and is aimed at platform administrators to assess OGX for enterprise production deployments.

The reference guidelines include the following information:

- All network ports exposed by OGX components, including server, operator, sidecars, etc.
- The Transport Layer Security (TLS) chain from an external client to an OGX server and FIPS cipher enforcements.
- Authentication mechanisms and their trust models.
- Authorization models at the route-level, resource-level ABAC, and tenant isolation.
- Secrets inventory, provisioning and rotation.

Each section provides a detailed reference for your OGX deployments.

3.1. NETWORK EXPOSURE

The following documentation provides a reference for OGX networking configurations.

Ingress ports

Table 3.1. Ingress ports details

Port	Protocol	Component	Purpose	Configurable
443/TC P	HTTPS	Gateway API, Envoy	External client access with TLS termination	Platform-level
8321/TC P (default)	HTTP	OGX Server	API server, cluster-internal only	Yes – spec.network.port (1-65535, default 8321)
Configur able	HTTP	OGX Server	Prometheus metrics only when monitoring is configured	Yes – spec.monitoring.metr icsPort
9443/T CP	HTTPS	OGX Operator, webhook	ValidatingWebhookConfiguration for OGXServer CRD validation	No
8080/T CP	HTTP	OGX Operator	Prometheus metrics, localhost-bound	No
8081/TC P	HTTP	OGX Operator	Health/readiness probes	No

Egress destinations

OGX pods initiate connections to remote providers and storage backends. No egress occurs unless a provider or storage type is configured. Destinations can be specified in the following ways:

- **CRD fields** – For example: **spec.providers.inference.remote.vllm[].endpoint**.
- **Environment variables** – The default OGX distribution **config.yaml** file uses **\${env.VLLM_URL}**, **\${env.OPENAI_BASE_URL}**, etc., which are set with Secrets or ConfigMaps CRs.
- **Custom config override** – A full **config.yaml** supplied via **spec.overrideConfig**, where endpoints are specified directly in the provider configuration.

Table 3.2. Egress destinations for each provider

Destination	Protocol	Default Port	Configuration
vLLM endpoint	HTTPS	User-supplied	spec.providers.inference.remote.vllm[].endpoint
OpenAI API	HTTPS	443, api.openai.com	spec.providers.inference.remote.openai[].endpoint
Azure OpenAI	HTTPS	User-supplied	spec.providers.inference.remote.azure[].endpoint
AWS Bedrock	HTTPS (SigV4)	443 (region-derived)	spec.providers.inference.remote.bedrock[].region
WatsonX	HTTPS	User-supplied	spec.providers.inference.remote.watsonx[].endpoint
Google Vertex AI	HTTPS	443 (project/location-derived)	spec.providers.inference.remote.vertexai[]
PostgreSQL (Pgvector)	TCP (pg wire)	5432	spec.providers.vectorlo.remote.pgvector[].host, .port
Milvus	gRPC / HTTPS	User-supplied	spec.providers.vectorlo.remote.milvus[].uri
Qdrant	REST or gRPC	User-supplied	spec.providers.vectorlo.remote.qdrant[].url, .host, .port, .grpcPort
Brave Search API	HTTPS	443 (SaaS)	spec.providers.toolRuntime.remote.braveSearch[] (no endpoint override)
Tavily Search API	HTTPS	443 (SaaS)	spec.providers.toolRuntime.remote.tavilySearch[] (no endpoint override)

Destination	Protocol	Default Port	Configuration
S3 (file storage)	HTTPS	443 or custom	spec.providers.files.remote.s3.endpointUrl
Docling Serve	HTTPS	User-supplied	spec.providers.fileProcessors.remote.doclingServe.baseUrl
Redis (KV storage)	TCP (Redis)	6379 (typical)	spec.storage.kv.endpoint
PostgreSQL (SQL storage)	TCP (pg wire)	Via connection string	spec.storage.sql.connectionString

The "Configuration" column displays CRD field paths. When using the default distribution config, the same destinations are set via environment variables:

Table 3.3. Environment variable fields

Destination	Environment variable
vLLM	VLLM_URL
OpenAI	OPENAI_BASE_URL
Azure OpenAI	AZURE_API_BASE
AWS Bedrock	AWS_DEFAULT_REGION
WatsonX	WATSONX_BASE_URL
Pgvector	PGVECTOR_HOST, PGVECTOR_PORT
Milvus	MILVUS_ENDPOINT
Qdrant	QDRANT_URL, QDRANT_HOST, QDRANT_PORT
S3	S3_ENDPOINT_URL
PostgreSQL	POSTGRES_HOST, POSTGRES_PORT

Table 3.4. Internal communication paths

Source	Destination	Protocol	Port	Purpose
Gateway / Ingress	OGX Server Pod	HTTP	spec.network.port , default 8321	API request forwarding via ClusterIP Service
Prometheus	OGX Server Pod	HTTP	spec.monitoring.metricsPort , if configured	Metrics scraping via ServiceMonitor
Kubernetes API Server	OGX Operator, webhook	HTTPS	9443	OGXServer CRD validation, via webhook Service on port 443
OGX Operator	Kubernetes API	HTTPS	443 , api-server	Reconciliation, status updates, RBAC checks
OGX Operator	OGX Server Pod	HTTP	spec.network.port	Provider and version status probes

3.2. TLS GUIDELINES

The Transport Layer Security (TLS) chain to the OGX Server from an external client has three layers.

- Layer 1 - External TLS, also known as Client to Gateway**The Gateway API terminates TLS at the envoy ingress. Certificate management and cipher configurations are managed at the platform level.
- Layer 2 - (Optional) Server-level TLS, also known as east to west**The OGX Server supports TLS for east-west encryption: mTLS between gateway and backend and for non-gateway deployments. When TLS is configured with **tls_certfile** and **tls_keyfile** parameters on the **ServerConfig** CR, the server enforces FIPS-approved cipher suites only.
- Layer 3 – Outbound TLS, OGX to external services:**Connections to cloud provider APIs including OpenAI, Azure, Bedrock, WatsonX, use TLS. Connections to self-hosted inference endpoints, including vLLM and Ollama, may use TLS depending on deployment configuration. Custom CA bundles can be configured for self-signed certificates.

3.2.1. Server-level TLS and FIPS cipher enforcement

When TLS is enabled on the OGX Server, only FIPS 140-3 approved cipher suites are permitted. The server validates configured ciphers against an allowlist and raises a hard error if non-approved ciphers are specified. OpenShift AI requires minimum TLS version TLS 1.2. The minimum is enforced by Python 3.10+ defaults and the server does not support TLS 1.0 or 1.1.

TLS is required by default. The OGX server refuses to start unless TLS certificates, **tls_certfile** and **tls_keyfile**, are configured or the **--insecure** flag is passed. Without either, **ogx stack run** exits with the following error: "TLS required: set tls_certfile/tls_keyfile in server config or pass **--insecure** to disable."

Enforcement behavior:

- If TLS is configured but no ciphers are specified, the FIPS-approved list is applied automatically.
- If TLS is configured with explicit ciphers, each cipher is validated against the approved list. Invalid ciphers cause a startup error.
- If **--insecure** is set, TLS and cipher validation are bypassed entirely and the server runs plaintext HTTP.

Table 3.5. Supported cipher suites

Cipher suite	Key exchange	Authentication	Encryption	MAC
ECDHE-ECDSA-AES128-GCM-SHA256	ECDHE	ECDSA	AES-128-GCM	SHA-256
ECDHE-RSA-AES128-GCM-SHA256	ECDHE	RSA	AES-128-GCM	SHA-256
ECDHE-ECDSA-AES256-GCM-SHA384	ECDHE	ECDSA	AES-256-GCM	SHA-384
ECDHE-RSA-AES256-GCM-SHA384	ECDHE	RSA	AES-256-GCM	SHA-384
DHE-RSA-AES128-GCM-SHA256	DHE	RSA	AES-128-GCM	SHA-256
DHE-RSA-AES256-GCM-SHA384	DHE	RSA	AES-256-GCM	SHA-384

3.2.2. CA bundle configuration

The OGX operator manages custom CA bundles for outbound TLS connections to external services with self-signed or internal CA certificates.

The CA bundle is configured in the **spec.tls.trust.caCertificates** on the OGXServer CRD.

Security controls on CA bundle reference

- Maximum bundle size: 10MB. This maximum prevents resource exhaustion.
- Maximum certificate count: 1000.
- PEM format validation before processing. Only valid X.509 CERTIFICATE blocks are extracted.
- Validation errors prevent deployment with clear error messages in CR status.
- Source ConfigMaps must be same-namespace, cross-namespace references are not supported.

Example caCertificates configuration

```
spec:
  tls:
    trust:
      caCertificates:
        - name: odh-trusted-ca-bundle
          key: ca-bundle.crt      # CNO-injected cluster CAs
        - name: odh-trusted-ca-bundle
          key: odh-ca-bundle.crt  # User-specified custom CAs
```

CA lifecycle:

1. The platform administrator creates a **ConfigMap** containing PEM-encoded CA certificates in the same namespace as the OGXServer, with label **ogx.io/watch: "true"**.
2. The operator controller reads and validates certificates from the source **ConfigMaps**.
3. Valid certificates are concatenated into a single PEM file.
4. The operator creates a managed **ConfigMap** called **{instance-name}-ca-bundle**.
5. The managed **ConfigMap** is mounted at **/etc/ssl/certs/ca-bundle/ca-bundle.crt**.
6. **SSL_CERT_FILE** is set automatically to point to the mounted bundle.
7. The Pods restart automatically when the source CA bundle **ConfigMap** changes.

3.3. AUTHENTICATION GUIDELINES

OGX supports two authentication modes depending on the deployment topology.

3.3.1. Option 1: Direct token validation

In this option, the OGX server validates bearer tokens directly against a designated authorization provider. The default OGX distribution uses the **oauth2_token** provider for JWKS-based verification—communicating with the JWKS endpoint to extract user principals, access attributes, and tenant context straight from the JWT claims.

Example spec in the OGX distribution

```
server:
  auth:
    provider_config:
      type: ${env.AUTH_ISSUER:+oauth2_token}
      audience: ${env.AUTH_AUDIENCE:=ogx}
      issuer: ${env.AUTH_ISSUER:=}
    jwks:
      uri: ${env.AUTH_JWKS_URI:=}
      key_recheck_period: ${env.AUTH_JWKS_RECHECK_PERIOD:=3600}
      verify_tls: ${env.AUTH_VERIFY_TLS:=true}
```

While a gateway may handle TLS termination and request routing, the authentication logic is managed end-to-end by the OGX server rather than the header injection.

3.3.2. Option 2: Gateway-delegated authentication

Edge security concerns—specifically token validation—are fully offloaded to the gateway layer. The gateway authenticates incoming clients against the identity provider and injects trusted identity claims into the request headers. The OGX server's **UpstreamHeaderAuthProvider** then consumes these headers directly, trusting them without additional validation.

Trust boundary invariants for `upstream_header` mode

The following safeguards are mandatory when using header-based authentication, called **`upstream_header`**. They do not apply to providers that perform direct validation, such as **`oauth2_token`**.

1. **Gateway-Only Reachability:** OGX must never be directly reachable by clients when using **`upstream_header`** authentication.
The **UpstreamHeaderAuthProvider** trusts gateway-injected headers without secondary validation. Direct client access allows attackers to forge identity headers and impersonate any user. Operators must implement per-instance **NetworkPolicies** restricting ingress strictly to same-namespace pods and explicitly authorized namespaces.
2. **Upstream Header Stripping:** The gateway must strip all incoming client-supplied identity headers before forwarding requests to OGX.
This includes principal, tenant, and attribute headers. Without strict stripping at the edge, external clients can inject arbitrary identity claims by pre-populating headers the gateway is expected to set.
3. **Auth-Resolved Tenant Context:** Tenant partitioning must strictly use the **`tenant_id`** resolved by the authentication provider from trusted sources.
Valid sources include gateway headers, verified JWT claims, or auth endpoint responses. Tenant context must **never** be derived from client-controlled request payloads, search parameters, or **`X-OGX-Provider-Data`** headers.



NOTE

This invariant applies to both header-based and direct-validation authentication models.

Architecture topology

Approved deployment topology (`upstream_header`)

Client —► Gateway (validates token, strips identity headers, injects x-user-id, x-tenant-id, x-user-roles) —► OGX Server

Non-compliant topology (Identity spoofing risk)

Client —► OGX Server (UpstreamHeaderAuthProvider trusts any x-user-id, x-tenant-id headers — client can forge identity)

3.3.3. Authentication providers

OGX supports several authentication providers tailored to different deployment and validation strategies:

Table 3.6. Supported authentication providers

Provider	Token Source	Validation	Tenant Resolution	Use Case
oauth2_token	Bearer header	JWKS or RFC 7662 introspection	JWT claim (tenant_claim)	Standard production (with or without gateway)
kubernetes	Bearer header	K8s SelfSubjectReview API	K8s user info claim (tenant_claim)	Kubernetes-native deployments
upstream_header	Gateway-injected headers	None (trusts headers)	Dedicated header (tenant_header)	Production with gateway
github_token	Bearer header	GitHub API /user endpoint	N/A	Development, GitHub-based auth
custom	Bearer header	External endpoint (POST)	Response field (tenant_field)	Custom auth integrations

Supported algorithms The `oauth2_token` provider uses a static allowlist of asymmetric-only algorithms. Symmetric algorithms (HS256, HS384, HS512) are explicitly excluded to prevent algorithm confusion attacks where an attacker tricks the server into using the public key as an HMAC secret.

Table 3.7. Supported algorithms list

Family	Algorithms
RSA PKCS#1 v1.5	RS256, RS384, RS512
RSA-PSS	PS256, PS384, PS512
ECDSA	ES256, ES384, ES512

The OAuth2 Token Provider (**oauth2_token**)

The **oauth2_token** provider exclusively permits asymmetric signature algorithms. Symmetric algorithms, such as HS256, HS384, and HS512, are disabled to eliminate algorithm confusion attacks, preventing malicious actors from using public keys as shared HMAC secrets.

Table 3.8. OAuth2 Token Provider details

Field	Type	Default	Description
jwtks.uri	string	Required if no introspection	JWKS endpoint for public key retrieval
jwtks.token	string	null	Bearer token for authenticated JWKS access

Field	Type	Default	Description
jwks.key_recheck_period	int	3600	Seconds between JWKS key refresh
introspection.url	string	Required if no JWKS	RFC 7662 token introspection endpoint
introspection.client_id	string	Required	Client ID for introspection
introspection.client_secret	SecretString	Required	Client secret for introspection
introspection.send_secret_in_body	bool	false	Send credentials in body vs. Basic auth header
audience	string	ogx	Expected aud claim in JWT
issuer	string	null	Expected iss claim – validated if set
claims_mapping	dict	{sub: roles, username: roles, groups: teams, team: teams, project: projects, tenant: namespaces, namespace: namespaces}	Maps JWT claims to access control attributes
tenant_claim	string	null	JWT claim to extract as tenant_id
verify_tls	bool	true	TLS verification for JWKS/introspection endpoints
tls_cafile	path	null	Custom CA certificate file

The Kubernetes Provider

The Kubernetes provider allows OGX to delegate token verification directly to the underlying cluster control plane using standard ServiceAccount tokens.

Table 3.9. Kubernetes provider details

Field	Type	Default	Description
api_server_url	string	"https://kubernetes.default.svc"	Kubernetes API server URL

Field	Type	Default	Description
verify_tls	bool	true	TLS verification for API server
tls_cafile	path	null	Custom CA certificate file
claims_mapping	dict	{username: roles, groups: roles}	Maps K8s user info to access attributes
tenant_claim	string	null	K8s claim to extract as tenant_id

The upstream header provider (**upstream_header**)

The **upstream_header** provider offloads the CPU cost of token validation entirely to an ingress gateway.

Table 3.10. Upstream header provider details

Field	Type	Default	Description
principal_header	string	Required	HTTP header containing authenticated user ID
tenant_header	string	null	HTTP header containing tenant ID
attributes_header	string	null	Deprecated. Header with JSON-encoded attribute dict
attribute_headers	dict	null	Maps header names to attribute categories (preferred over attributes_header)

The GitHub token provider (**github_token**)

The **github_token** provider is a lightweight option for local development, staging environments, or internal developer tools integrated with GitHub.

Table 3.11. GitHub token provider details

Field	Type	Default	Description
github_api_base_url	string	"https://api.github.com"	GitHub API base URL (override for GHE)
claims_mapping	dict	{login: roles, organizations: teams}	Maps GitHub user fields to access attributes

The custom provider (**custom**)

The **custom** provider integrates with legacy identity systems or proprietary internal authentication services without modifying OGX core code.

Table 3.12. Custom provider details

Field	Type	Default	Description
endpoint	string	Required	URL of the external auth endpoint (POST)
tenant_field	string	null	Field in auth response to extract as tenant_id

3.3.4. UpstreamHeaderAuthProvider trust model

The **UpstreamHeaderAuthProvider** is the authentication provider for gateway-delegated deployments, where an upstream gateway, for example Authorino, Istio, or similar, handles authentication. It is not the default RHOAI configuration, but it is available for deployments that prefer centralized gateway authentication.

Security implications:

In this architecture, OGX relies entirely on the perimeter gateway layer to authenticate requests, sanitize incoming headers, and inject verified identity context.

Mandatory Gateway Pipeline: . Authenticate incoming client requests against the configured Identity Provider. . Strip all client-supplied identity headers prior to upstream forwarding. . Inject verified identity headers into the request context before forwarding to OGX.

This design is dangerous when OGX is directly client-reachable, because any client can set arbitrary identity headers and impersonate any user or tenant.

Configuration example:

```
server:
  auth:
    provider_config:
      type: upstream_header
      principal_header: x-user-id
      tenant_header: x-tenant-id
      attribute_headers:
        x-user-roles: roles
        x-user-teams: teams
        x-user-namespaces: namespaces
```

where

- **principal_header** – Specifies the authenticated user identity, required parameter.
- **tenant_header** – Specifies the tenant partition key, optional parameter.

Required safeguards

- NetworkPolicy must prevent direct client access to the OGX server pod

- Gateway must strip x-user-id, x-tenant-id, x-user-roles, x-user-teams, x-user-namespaces from incoming client requests
- Gateway must inject these headers only after successful authentication

3.4. AUTHORIZATION GUIDELINES

OGX enforces authorization at three independent layers, evaluated in order:

Request —► Route Authorization —► Resource ABAC —► Tenant Isolation

3.4.1. Route-level authorization

The **RouteAuthorizationMiddleware** parameter evaluates route-level access rules before the request reaches any resource logic.

Role structure

```
server:
  auth:
    route_policy:
      # Admins can access all routes
      - permit:
          paths: "*"
          when: "user with admin in roles"
          description: "admins have access to all routes"

      # Developers can access chat and inference
      - permit:
          paths:
            - "/v1/chat/completions"
            - "/v1/responses*"
          when: "user with developer in roles"
          description: "developers can use inference APIs"

      # Block everything else
      - forbid:
          paths: "*"
          description: "deny all unmatched routes"
```

If no **route_policy** is configured, all routes are allowed.

3.4.2. Resource-level ABAC

After route authorization, the **is_action_allowed()** function evaluates resource-level access using Attribute-Based Access Control (ABAC).

The OGX distribution ships with an explicit **access_policy** that is more restrictive than the code-level fallback. This policy defines the following: anyone can read system-registered resources, including models and built-in tool groups. Any authenticated user can create resources, but only the owner can read, update, or delete user-created resources. There is no attribute-based sharing in this distribution.

Default policy in the OGX distribution

–

```

access_policy:
- permit:
  actions: [read]
  when: resource is unowned
  description: "All users can read system resources"
- permit:
  actions: [create]
  description: "Authenticated users can create resources"
- permit:
  actions: [read, update, delete]
  when: user is owner
  description: "Owners can manage their own resources"

```

You can customize the **access_policy** parameters as well.

Example custom policy

```

server:
  auth:
    access_policy:
      - permit:
        actions: [create, read, update, delete]
        resource: "model:.*"
        when: "user with admin in roles"
        description: "admins have full access to all models"

      - permit:
        actions: [read]
        resource: "model:.*"
        when: "user in owners teams"
        description: "team members can read team models"

      - forbid:
        actions: [create, delete]
        resource: "vector_store:.*"
        unless: "user with admin in roles"
        description: "only admins can create or delete vector stores"

```

3.4.3. Tenant isolation

Tenant isolation is the outermost authorization layer and is evaluated before ABAC. The **TenancyMiddleware** and **AuthorizedSqlStore** parameters enforce tenant boundaries.

Table 3.13. Tenant isolation modes

Mode	Behavior	Use case
disabled (default)	No tenant enforcement.	Development, single-tenant, and ABAC-only deployments
single	All resources belong to one configured tenant. default_tenant_id is injected into every request.	Single-tenant production with explicit tenant tagging

Mode	Behavior	Use case
multi	Full tenant isolation. Every request must resolve a tenant_id from authentication. Requests without a tenant are rejected.	Multi-tenant production

OGX enforces tenant isolation through the following mechanisms:

- **Hard partitioning** – **AuthorizedSqlStore** appends a **WHERE tenant_id = ?** clause to every query before evaluating ABAC policies.
- **Default deny** – In **multi** mode, if a request has no **tenant_id**, it generates a **WHERE 1=0** predicate, ensuring the query returns zero records by default.
- **Intra-tenant ABAC** – Attribute-Based Access Control policies are evaluated only after tenant boundary filtering has succeeded.

Table 3.14. Tenant isolation resolution by auth provider

Auth Provider	Tenant Source	Configuration
upstream_header	Dedicated header	tenant_header: x-tenant-id
oauth2_token	JWT claim	tenant_claim: tenant
kubernetes	K8s user info claim	tenant_claim: namespace
custom	Auth endpoint response field	tenant_field: tenant_id
None, dev mode	Server config default	default_tenant_id in tenancy config

3.4.4. Access policy configuration reference

Actions The **Action** enum defines the four CRUD operations used in access control rules:

Table 3.15. Resource action definitions

Action	Description
create	Register a new resource
read	Retrieve or list resources
update	Modify an existing resource
delete	Remove a resource

AccessRule (Resource-Level)

Each **AccessRule** defines a permit or forbid scope with optional conditions. Rules are evaluated in declaration order: first-match priority, default deny.

Table 3.16. Policy evaluation rule fields

Field	Type	Required	Validation
permit	Scope	One of permit/forbid	Mutually exclusive with forbid
forbid	Scope	One of permit/forbid	Mutually exclusive with permit
when	string list[string]	No	Mutually exclusive with unless
unless	string list[string]	No	Mutually exclusive with when
description	string	No	Used in audit logs

Scope

Table 3.17. Scope fields

Field	Type	Default	Description
principal	string	null (matches all)	Exact match on user principal
actions	Action list[Action]	(required)	Action(s) this rule applies to
resource	string	null (matches all)	Resource pattern – exact (model::my-model), wildcard (model::*), or regex (regex:model::(llama mistral)-.*)

RouteAccessRule (Route-Level)

The **RouteAccessRule** structure is similar to the **AccessRule**, but scoped to API routes instead of resources.

Table 3.18. RouteAccessRule fields

Field	Type	Required	Validation
-------	------	----------	------------

Field	Type	Required	Validation
permit	RouteScope	One of permit/forbid	Mutually exclusive with forbid
forbid	RouteScope	One of permit/forbid	Mutually exclusive with permit
when	string list[string]	No	Mutually exclusive with unless
unless	string list[string]	No	Mutually exclusive with when
description	string	No	Used in audit logs

RouteScope

Table 3.19. Route scope fields

Field	Type	Description
paths	string list[string]	Path pattern(s): exact (" v1/chat/completions "), prefix wildcard (" v1/files* "), full wildcard (" * "), or regex ("regex:v1/(chat

Condition Grammar

The **when** and **unless** fields accept the following condition expressions:

Table 3.20. ABAC condition expressions

Expression	Description
user is owner	User principal matches resource owner principal (same tenant)
user is not owner	User is not the resource owner
user with <value> in <attribute>	User has <value> in their <attribute> list
user with <value> not in <attribute>	User does not have <value> in their <attribute> list
user in owners <attribute>	User shares a value in <attribute> with the resource owner

Expression	Description
user not in owners <attribute>	User does not share a value in <attribute> with the resource owner
resource is unowned	Resource has no owner

3.5. SECRETS MANAGEMENT

Every secret required by the OGX deployment is referenced using **SecretKeyRef** in the **OGXServer** Custom Resource Definition (CRD). The operator automatically injects them as environment variables into the OGX server container. To ensure proper discovery and caching by the operator, each referenced Kubernetes Secret must reside in the same namespace as the **OGXServer** instance and carry the **ogx.io/watch: "true"** label.

Table 3.21. Secret configuration inventory

Provider category	CRD field path	Secret field	Environment variable pattern	Required
Inference - vLLM	spec.providers.inference.remote.vllm[].apiToken	API token	OGX_<ID>_API_TOKEN	Optional
Inference - OpenAI	spec.providers.inference.remote.openai[].apiKey	API key	OGX_<ID>_API_KEY	Required
Inference - Azure	spec.providers.inference.remote.azure[].apiKey	API key	OGX_<ID>_API_KEY	Required
Inference - Bedrock	spec.providers.inference.remote.bedrock[].apiKey	API key	OGX_<ID>_API_KEY	Optional
Inference - Bedrock	spec.providers.inference.remote.bedrock[].awsAccessKeyId	AWS access key	OGX_<ID>_AWS_ACCESS_KEY_ID	Optional
Inference - Bedrock	spec.providers.inference.remote.bedrock[].awsSecretAccessKey	AWS secret key	OGX_<ID>_AWS_SECRET_ACCESS_KEY	Optional
Inference - Bedrock	spec.providers.inference.remote.bedrock[].awsSessionToken	AWS session token	OGX_<ID>_AWS_SESSION_TOKEN	Optional
Inference - WatsonX	spec.providers.inference.remote.watsonx[].apiKey	API key	OGX_<ID>_API_KEY	Required

Provider category	CRD field path	Secret field	Environment variable pattern	Required
VectorIO - Pgvector	spec.providers.vectorlo.remote.pgvector[].password	DB password	OGX_<ID>_PASSWORD	Required
VectorIO - Milvus	spec.providers.vectorlo.remote.milvus[].token	Auth token	OGX_<ID>_TOKEN	Optional
VectorIO - Qdrant	spec.providers.vectorlo.remote.qdrant[].apiKey	API key	OGX_<ID>_API_KEY	Optional
ToolRuntime - BraveSearch	spec.providers.toolRuntime.remote.braveSearch[].apiKey	API key	OGX_<ID>_API_KEY	Required
ToolRuntime - TavilySearch	spec.providers.toolRuntime.remote.tavilySearch[].apiKey	API key	OGX_<ID>_API_KEY	Required
Files - S3	spec.providers.files.remote.s3.awsAccessKeyId	AWS access key	OGX_REMOTE_S3_AWS_ACCESS_KEY_ID	Optional
Files - S3	spec.providers.files.remote.s3.awsSecretAccessKey	AWS secret key	OGX_REMOTE_S3_AWS_SECRET_ACCESS_KEY	Optional
FileProcessors - DoclingServe	spec.providers.fileProcessors.remote.doclingServe.apiKey	API key	OGX_<ID>_API_KEY	Optional
Storage - Redis (KV)	spec.storage.kv.password	Redis password	OGX_STORAGE_KV_PASSWORD	Optional
Storage - PostgreSQL (SQL)	spec.storage.sql.connectionString	Connection string	OGX_STORAGE_SQL_CONNECTION_STRING	Required when type=postgres

3.5.1. Operator secret collection mechanism

The operator walks the OGXServer spec at reconciliation time to collect all secret references:

1. **CollectSecretRefs** calls **collectSecretRefsRaw()** which walks each provider category: inference, vectorlo, toolRuntime, files, batches, responses, fileProcessors, and storage.
2. Environment variable naming follows the pattern **OGX_<PROVIDER_ID>_<FIELD>**:
 - a. **buildEnvVarName**, with the **providerID** and **field** parameters, constructs the name.
 - b. **normalizeEnvVarField()** uppercases the provider ID, replaces non-alphanumeric characters with `_`, collapses consecutive underscores, and trims leading/trailing underscores.
 - c. Example: provider ID **remote-openai** + field **API_KEY** → **OGX_REMOTE_OPENAI_API_KEY**
3. The Config injection uses **envVarRef**, with the **providerID** and **field** parameters, to produce **#{env.OGX_<ID>_<FIELD>}** substitution strings in the generated **config.yaml**. The OGX server resolves these at startup.
4. Then de-duplication occurs: When multiple providers reference the same secret, the first seen environment variable duplicates are silently dropped.
5. Collision detection: **ValidateSecretRefEnvVarNames()** runs during webhook validation. If two different providers would produce the same env var name pointing to different secrets, the CRD update is rejected.

3.5.2. Provisioning and rotation

Provisioning a Secret CR example

```
apiVersion: v1
kind: Secret
metadata:
  name: <my-openai-creds>
  namespace: ogx-system      # must match OGXServer namespace
  labels:
    ogx.io/watch: "true"     # required for operator cache detection
type: Opaque
data:
  api-key: <base64-encoded-key>
```

where:

<my-openai-creds>

Specifies the name of the Secret. Must be a minimum of 1 character and a valid Kubernetes Secret name.

<base64-encoded-key>

Specifies the base64-encoded API key. Must be a minimum of 1 character, maximum of 253 characters, and match `^[a-zA-Z0-9]([a-zA-Z0-9\-_]*[a-zA-Z0-9])?$.`

Rotation workflow:

1. Update the Kubernetes Secret data, for example **kubectl patch, secret, my-openai-creds**.
2. The operator detects the **Secret** change via the watched cache

3. The operator reconciles the OGXServer Deployment – environment variable bindings use `secretKeyRef`, so the kubelet injects updated values
4. The Deployment rolls pods with updated secret values

Secrets with the **ogx.io/watch: "true"** label trigger immediately. Secrets without this label are read via a non-cached API client with eventual consistency.



IMPORTANT

Never store secrets in the **OGXServer** CR spec itself, you need to always use **SecretKeyRef** references to Kubernetes Secrets. The operator never reads secret data directly. The operator only passes references to the kubelet for injection.

3.6. NETWORKPOLICY CONFIGURATION

[role="__abstract"]

The OGX operator creates a per-instance `NetworkPolicy` for each `OGXServer`. By default, only ingress is enforced. The `NetworkPolicy` uses `podSelector` matching on app: `ogx` and `app.kubernetes.io/instance: <instance-name>` to target only that instance's pods.

3.6.1. Default NetworkPolicy rules

Table 3.22. Default ingress peers

Peer	Selector	Purpose
Same-namespace pods	podSelector: {}	Allow all pods in the OGXServer's namespace
Operator namespace	namespaceSelector: {matchLabels: {kubernetes.io/metadata.name: <operator-ns>}}	Allow operator to reach OGX for status probes
OpenShift router	namespaceSelector: {matchLabels: {network.openshift.io/policy-group: ingress}}	Allow ingress controller traffic, only when spec.network is configured

The following peer is enabled only when **spec.monitoring.metricsPort > 0**

Table 3.23. Monitoring ingress rule

Peer	Port	Purpose
Monitoring namespaces	Metrics port (TCP)	Allow Prometheus scraping from namespaces with network.openshift.io/policy-group: monitoring

3.6.2. Custom network access rules

The CRD **spec.network.policy** provides full control over NetworkPolicy behavior:

Field	Type	Default	Description
spec.network.policy.enabled	bool	true	Set false to disable the NetworkPolicy entirely
spec.network.policy.ingress	[]NetworkPolicyIngressRule	Generated defaults	Custom ingress rules – replaces all defaults when specified
spec.network.policy.egress	[]NetworkPolicyEgressRule	None	Custom egress rules – adds Egress to policyTypes
spec.network.policy.policyTypes	[]PolicyType	["Ingress"]	Enforced policy types

- When custom ingress rules are provided, the operator does not combine them with defaults. The custom rules completely replace the generated rules. This gives the deployer full control, however the deployer must explicitly include same-namespace and operator-namespace peers if those are still needed.
- When egress rules are provided, the operator automatically injects a **kube-dns** egress rule to ensure pods can resolve DNS. The **policyTypes** field is automatically set to **["Ingress", "Egress"]**.

Example YAML to restrict ingress to a specific namespace:

```
spec:
  network:
    policy:
      ingress:
        - from:
            - namespaceSelector:
                matchLabels:
                  kubernetes.io/metadata.name: my-gateway-namespace
          ports:
            - protocol: TCP
              port: 8321
```

OpenShift AI specific rules

Label	Value	Purpose
network.openshift.io/policy-group	ingress	Identifies router/ingress controller namespaces. Included as default peers when spec.network is configured

Label	Value	Purpose
network.openshift.io/policy-group	monitoring	Identifies monitoring namespaces, for example openshift-monitoring . Allowed to scrape the metrics port when monitoring is enabled

3.7. GUARDRAILS INTEGRATION SECURITY

OGX has two layers of guardrails available in OpenShift AI deployments:

- The OGX built-in moderation endpoint integration.
- RHOAI NeMo Guardrails platform integration.

3.7.1. OGX Responses API guardrails

OGX's built-in guardrails use a **moderation_endpoint** configuration pointing at any OpenAI-compatible **/v1/moderations** endpoint. This is configured on the responses provider, rather than CRD level.

Table 3.24. Example configurations

Field	Type	Description
moderation_endpoint	string	URL of an OpenAI-compatible /v1/moderations endpoint
moderation_headers	dict	HTTP headers for authentication, for example Authorization: Bearer sk-.... Server-side only, never exposed to clients.

Clients opt in per-request by setting **guardrails: true** on the Responses API request. If **guardrails** is set to **true** but no **moderation_endpoint** is configured, the server returns a 400 error instructing the client to contact their platform administrator.

Fail-closed behavior The **run_guardrails()** function is designed to fail closed: any error blocks content rather than allowing it through:

- HTTP errors or timeouts → content blocked with "moderation service unavailable" error.
- Invalid JSON response → content blocked with "invalid response" error.
- Missing or malformed results array → content blocked with "unexpected format" error.
- Flagged: true in any result → content blocked with flagged category names
- **Only when no errors occur and no result is flagged → content passes**

The moderation endpoint connection uses the system CA trust store or the CA bundle configured via `spec.tls.trust.caCertificates`. Authentication is handled via **moderation_headers**.

3.7.2. RHOAI NeMo guardrails integration

The following configuration is the recommended guardrails path for OpenShift AI production environments.

NeMo Guardrails is managed with the following parameters:

- The **NemoGuardrails** CRD, **trustyai.opendatahub.io/v1alpha1**: Defines guardrails configurations, rails, and model references.
- The **MCPGuardrailsMode** toggle on the TrustyAI component spec: Enables the mcp-guardrails overlay in the opendatahub-operator
- **odh-dashboard** gen-ai Backend-for-frontend (BFF): Provides a client calling the NeMo Guardrails **/v1/guardrail/checks** endpoint with the inline guardrail configuration.

3.8. HARDENING RECOMMENDATIONS

Table 3.25. Deployment hardening checklist

Category	Action
Authentication	Configure an auth provider (oauth2_token or upstream_header behind a gateway)
Authentication	Ensure FIPS-approved JWT algorithms only (no HS*)
Gateway Trust	If using upstream_header , verify gateway-only reachability via NetworkPolicy
Gateway Trust	If using upstream_header , configure gateway to strip identity headers before forwarding
TLS	Verify --insecure flag is not used – TLS must be enforced in production
TLS	Enable TLS on OGX server or terminate at gateway
TLS	Configure CA bundle for custom CAs
Authorization	Configure route_policy for API-level access control
Authorization	Configure access_policy for resource-level ABAC
Tenancy	Enable multi-tenancy mode for multi-tenant deployments
Secrets	Label all referenced Secrets with ogx.io/watch: "true"
NetworkPolicy	Verify operator-managed NetworkPolicy is enabled (default)

Category	Action
NetworkPolicy	Add egress rules for production, restrict outbound to known destinations
Guardrails	Configure moderation_endpoint for content safety
HSTS	Verify HSTS is active when TLS is enabled, default: 1 year
Monitoring	Verify operator metrics endpoint is localhost-bound, port 8080; consider enabling kube-rbac-proxy if authenticated access is needed

3.8.1. Container security context

All OGX workloads run with restricted security contexts. These settings are defaults in the operator and are not user-configurable, they cannot be weakened with the CRD.

OGX server container

```
securityContext:
  runAsNonRoot: true
  allowPrivilegeEscalation: false
  seccompProfile:
    type: RuntimeDefault
  capabilities:
    drop:
      - "ALL"
```

OGX server pod-level

```
securityContext:
  runAsNonRoot: true
  seccompProfile:
    type: RuntimeDefault
```

Operator kube-rbac-proxy sidecar

```
securityContext:
  allowPrivilegeEscalation: false
  capabilities:
    drop:
      - "ALL"
```

Table 3.26. RBAC least privilege

Resource	Verbs	Security Note
secrets	get, list, watch	Read-only – the operator never creates or modifies secrets, only reads references

Resource	Verbs	Security Note
configmaps, persistentvolumeclaims	create, get, list, patch, update, watch	Full lifecycle management for managed resources
deployments	create, delete, get, list, patch, update, watch	Full lifecycle for OGX server deployments
serviceaccounts, services	create, delete, get, list, patch, update, watch	Per-instance SA and ClusterIP service
networkpolicies, ingresses	create, delete, get, list, patch, update, watch	Network access management
horizontalpodautoscalers, poddisruptionbudgets	create, delete, get, list, patch, update, watch	Scaling and availability
servicemonitors, prometheusrules	create, delete, get, list, patch, update, watch	Monitoring integration
clusterrolebindings	delete, get, list	Cleanup only – no create (used for upgrade cleanup of legacy bindings)
clusterroles	get, list, watch	Read-only – used to verify existence
rolebindings	create, delete, get, list, patch, update, watch	Per-instance SCC binding
securitycontextconstraints	use (limited to nonroot-v2)	SCC usage restricted to non-root only
ogxservers, ogxservers/status, ogxservers/finalizers	Full CRUD	Primary CRD management

RBAC recommendations

- Do not grant the operator's ServiceAccount additional ClusterRoles beyond manager-role and proxy-role
- The operator does not need cluster-admin – reject any deployment that requests it
- Per-instance ServiceAccounts are scoped to the OGXServer's namespace

3.8.2. Network segmentation

Egress Restrictions

Disabled by default, the operator's default NetworkPolicy restricts ingress only. For production deployments, define outbound rules under **spec.network.policy.egress** to restrict connections to trusted provider endpoints. When egress rules are set, the operator automatically appends a **kube-dns** rule to preserve DNS resolution.

Cross-Namespace Ingress Rules:

- Operator namespace: Always permitted for status health probes.
- OpenShift router namespaces: Permitted when spec.network is defined.
- Other namespaces: Require custom ingress rules or peers.



NOTE

Custom ingress rules override defaults. Always re-include same-namespace and operator-namespace peers explicitly when adding custom rules.

3.8.3. Audit logging

Table 3.27. Authentication events

Event	Level	Fields Logged	Sensitive Data
Auth success	DEBUG	principal, attributes_count, tenant_id	Never logs tokens or attribute values – only the count.
Token validation failure	WARNING	error (message only)	Never logs the token itself.
Auth service unavailable	WARNING	error	N/A
Auth request timeout	WARNING	(none)	N/A
Unauthenticated access to public endpoint	DEBUG	path	N/A

Table 3.28. Authorization events

Event	Level	Fields Logged
Route authorization decision	DEBUG	decision (APPROVED/DENIED), user, route, rule_index, reason
Resource ABAC decision	DEBUG	decision_str, user_str, qualified_resource_id, action, index, reason

Parameters that are never logged

- Bearer tokens or API keys

- Full user attribute values (only counts)
- Secret values or credential content
- Request/response bodies

HSTS Configuration

When TLS is enabled on the OGX server, HTTP Strict Transport Security (HSTS) is automatically active.

Field	Type	Default	Description
hsts_max_age	int	31536000 , 1 year	HSTS max-age directive in seconds. Set to 0 to disable

The HSTS header includes includeSubDomains. This is configured at the OGX server level **ServerConfig** in **datatypes.py**, not at the CRD level.

CHAPTER 4. OGX APIS

You can use the following APIs from OGX for AI actions.

4.1. SUPPORTED OGX APIS IN OPENSIFT AI

4.1.1. File Processors API

- **Endpoint:** `/v1alpha/file-processors`.
- **Providers:** All file processor backends deployed through OpenShift AI.
- **Support level:** Developer Preview

The File Processors API converts various document types into vector-ready chunks using configurable extraction backends, including Docling, PyPDF, and others. You can upload a document in your file storage and the API returns structured chunks.

4.1.2. Datasets_IO API

- **Endpoint:** `/v1alpha/datasetio`.
- **Providers:** All dataset_io backends deployed through OpenShift AI.
- **Support level:** Technology Preview.

The Dataset_IO API manages the input and output of datasets and their content.

4.1.3. Inference API

- **Endpoint:** `/v1alpha/inference`.
- **Providers:** All inference backends deployed through OpenShift AI.
- **Support level:** Developer Preview.



WARNING

The majority of the Inference API is deprecated. The Inference providers use the Completions and Chat Completions APIs now.

The Inference API enables conversational, message-based interactions with models served by OGX in OpenShift AI.

4.1.4. Tool Runtime API

- **Endpoint:** `/v1/tool-runtime`.
- **Providers:** All tool runtime backends deployed through OpenShift AI.

- **Support level:** Developer Preview.

The Tool Runtime API allows a model to dynamically call a tool at runtime.

4.1.5. Vector_IO API

- **Endpoint:** `/v1/vector-io`.
- **Providers:** All `vector_io` backends deployed through OpenShift AI.
- **Support level:** Developer Preview.

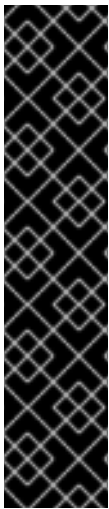
The Vector_IO API allows you to manage and query vector embeddings: numeric representations of data.

4.2. OPENAI-COMPATIBLE APIS IN OGX

OpenShift AI includes a OGX component that exposes OpenAI-compatible APIs. These APIs enable you to reuse existing OpenAI SDKs, tools, and workflows directly within your OpenShift environment, without changing your client code. This compatibility layer supports retrieval-augmented generation (RAG), inference, and embedding workloads by using OpenAI-compatible endpoints, schemas, and authentication patterns.

This compatibility layer has the following capabilities:

- **Standardized endpoints:** REST API paths align with OpenAI specifications.
- **Schema parity:** Request and response fields follow OpenAI data structures.



IMPORTANT

When connecting OpenAI SDKs or third-party tools to OpenShift AI, you must update the client configuration to use your deployment's OGX route as the **base_url**.

When you use OpenAI-compatible SDKs, the **base_url must include the /v1 path suffix** so that requests are routed to the OpenAI-compatible API surface exposed by OGX.

When you use OpenAI SDKs or send raw HTTP requests to OGX, always include the **/v1** path suffix in the base URL.

For example: **`http://ogx-service:8321/v1`**

Using the service endpoint without **/v1** results in request failures.

These endpoints are exposed under the OpenAI compatibility layer and are distinct from the native OGX APIs.

4.2.1. Supported OpenAI-compatible APIs in OpenShift AI

Before running the following examples, ensure you have:

- The OpenAI Python SDK installed: **`pip install -q openai rich`**
- A configured client pointing to your OGX endpoint

- Model IDs from your deployment (see Models API section)

```
from openai import OpenAI
import rich

# We'll be using a ogx server deployed in {productname-short}.
# Once all pods associated to the OGXServer are running,
# create the base_url using the ogx service hostname (with /v1 at the end when using openai sdk)
base_url = "http://ogx-distribution-service.my-project.svc.cluster.local:8321/v1"

client = OpenAI(
    api_key="your-ogx-key",
    base_url=base_url
)
```

For more information, see [Deploying a OGX server](#).

4.2.1.1. Models API

- **Endpoint:** `/v1/models`.
- **Providers:** All model-serving back ends configured within OpenShift AI.
- **Support level:** Technology Preview.

The Models API lists and retrieves available model resources from the OGX deployment running on OpenShift AI. By using the Models API, you can enumerate models, view their capabilities, and verify deployment status through a standardized OpenAI-compatible interface.

Example code in Python:

```
# List models available in the ogx server
models = client.models.list()
rich.print(models)

# Select the first LLM and first embedding model
model_id = next(m for m in models if m.custom_metadata["model_type"] == "llm").id
embedding_model_id = (
    em := next(m for m in models if m.custom_metadata["model_type"] == "embedding")
).id
embedding_dimension = em.custom_metadata["embedding_dimension"]
```

4.2.1.2. Chat Completions API

- **Endpoint:** `/v1/chat/completions`.
- **Providers:** All inference back ends deployed through OpenShift AI.
- **Support level:** Technology Preview.

The Chat Completions API enables conversational, message-based interactions with models served by OGX in OpenShift AI.

Example code in Python:

```
# Test chat completion functionality with a simple question
response = client.chat.completions.create(
    model=model_id,
    messages=[
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "What is the capital of France?"},
    ],
    temperature=0,
)
# Optional verification check
assert len(response.choices) > 0, "No response after basic inference on ogx server"
content = response.choices[0].message.content
rich.print(content)
```

4.2.1.3. Completions API

- **Endpoint:** `/v1/completions`.
- **Providers:** All inference back ends managed by OpenShift AI.
- **Support level:** Technology Preview.

The Completions API supports single-turn text generation and prompt completion.

Example code in Python:

```
# Test completion functionality with a simple question
response = client.completions.create(
    model=model_id,
    prompt="Answer with one word only: What is the capital of France?",
    max_tokens=64,
    temperature=0.1
)
# Optional verification check
assert len(response.choices) > 0, "No response after basic inference on ogx server"
content = response.choices[0].text
rich.print(content)
```

4.2.1.4. Embeddings API

- **Endpoint:** `/v1/embeddings`.
- **Providers:** All embedding models enabled in OpenShift AI.
- **Support level:** Technology Preview.

The Embeddings API generates numerical embeddings for text or documents that can be used in downstream semantic search or RAG applications.

Example code in Python:

```
# Create text embeddings
response = client.embeddings.create(
    input="Your text string goes here",
    model=embedding_model_id
```

```
)
embedding = response.data[0].embedding
rich.print(embedding[:5] + ["..."] + embedding[-5:])
```

4.2.1.5. Files API

- **Endpoint:** `/v1/files`.
- **Providers:** File system-based file storage provider for managing files and documents stored locally in your cluster.
- **Support level:** Technology Preview.

The Files API manages file uploads for use in embedding and retrieval workflows.

The Files API handles file storage only. Indexing files for retrieval requires a vector store, which is a separate provider managed through the Vector Stores API. The following example demonstrates a complete file-upload-and-index workflow that uses both APIs together.

Example code in Python:

```
import requests
from rich import print
from rich.rule import Rule
import time

# -----
# Download the PDF from url
# -----
print(Rule("[bold cyan]Downloading PDF[/bold cyan]"))

# We'll use IBM 2025-Q4 report to test RAG, as models don't have that info
pdf_url = "https://www.ibm.com/downloads/documents/us-en/1550f7eea8c0ded6"
filename = "ibm-Q4-2025-4q25-press-release.pdf"
title = "IBM-4Q25-Earnings-Press-Release"

print(" Fetching PDF from URL...")
response = requests.get(pdf_url)
response.raise_for_status()
print(" PDF fetched successfully")

print(f" Saving PDF as [bold]{filename}[/bold]...")
with open(filename, "wb") as f:
    f.write(response.content)
print(f" Downloaded and saved: [green]{filename}[/green]")

# -----
# Upload the PDF
# -----
print(Rule("[bold cyan]Uploading File[/bold cyan]"))

print(" 📁 Uploading file to Files API...")
with open(filename, "rb") as f:
    file_info = client.files.create(
        file=(filename, f),
        purpose="assistants"
```

```

)

print(" File uploaded successfully")
print(file_info)

# -----
# Create vector store
# -----
print(Rule("[bold cyan]Creating Vector Store[/bold cyan]"))

provider_id = "milvus-remote"

print(" Creating vector store with Milvus provider...")
vector_store = client.vector_stores.create(
    name="test_vector_store",
    extra_body={
        "embedding_model": embedding_model_id,
        "embedding_dimension": embedding_dimension,
        "provider_id": provider_id,
    },
)

print(" Vector store created")
print(vector_store)

# -----
# Add file to vector store
# -----
print(Rule("[bold cyan]Indexing File[/bold cyan]"))

print(" Adding uploaded file to vector store...")
vector_store_file = client.vector_stores.files.create(
    vector_store_id=vector_store.id,
    file_id=file_info.id,
    chunking_strategy={
        "type": "static",
        "static": {
            "max_chunk_size_tokens": 700,
            "chunk_overlap_tokens": 100,
        }
    },
    attributes={
        "title": title,
    },
)

print(" File added to vector store")
print(vector_store_file)

# -----
# Verify file is completed
# -----
print(Rule("[bold cyan]Waiting until file status is complete[/bold cyan]"))

# Wait for file processing to complete

```

```

print("Waiting for file processing to complete...")
max_wait_time = 300 # 5 minutes
start_time = time.time()

while time.time() - start_time < max_wait_time:
    files = client.vector_stores.files.list(vector_store_id=vector_store.id)
    if files.data:
        file_status = files.data[0].status
        print(f"File status: {file_status}")
        if file_status == "completed":
            print(" File processing completed!")
            break
        elif file_status == "failed":
            print("✗ File processing failed!")
            break
        time.sleep(5)
    else:
        print("⚠ Timeout waiting for file processing")

# Verify file is completed
files = client.vector_stores.files.list(vector_store_id=vector_store.id)
if files.data:
    print(f"\nFinal file status: {files.data[0].status}")
    print(f"File details: {files.data[0]}")
else:
    print("No files found in vector store")

print(Rule("[bold green]All tasks completed successfully ✓[/bold green]"))

```

4.2.1.6. Vector Stores API

- **Endpoint:** `/v1/vector_stores`.
- **Providers:** Remote vector store providers configured in OpenShift AI.
- **Support level:** Technology Preview.

The Vector Stores API manages the creation, configuration, and lifecycle of vector store resources in OGX. Through this API, you can create new vector stores, list existing ones, delete unused stores, and query their metadata, all using OpenAI-compatible request and response formats.

4.2.1.7. Vector Store Files API

- **Endpoint:** `/v1/vector_stores/{vector_store_id}/files`.
- **Providers:** Local inline provider configured for file storage and retrieval.
- **Support level:** Developer Preview.

The Vector Store Files API implements the OpenAI Vector Store Files interface and manages the association between document files and vector stores used for RAG workflows.

4.2.1.8. Responses API

- **Endpoint:** `/v1/responses`.

- **Providers:** All agents, inference, and vector providers configured in OpenShift AI.
- **Support level:** Generally Available

The Responses API generates model outputs by combining inference, file search, and tool-calling capabilities through a single OpenAI-compatible endpoint. It is particularly useful for retrieval-augmented generation (RAG) workflows that rely on the **file_search** tool to retrieve context from vector stores.

The Responses API orchestrates inference and retrieval but relies on separate providers for file storage (Files API) and vector indexing (Vector Stores API). The following example demonstrates a complete RAG workflow that uses the Files API, Vector Stores API, and Responses API together.

Example code in Python:

```
from rich import print
from rich.table import Table

system_instructions = """You are a financial document analysis assistant specialized in quarterly
earnings reports, annual filings, press releases, and earnings call transcripts.
You are designed to answer questions in a concise and professional manner.
Answer questions strictly using only the provided documents.
Base every answer strictly on the retrieved document content and cite the relevant section or excerpt
ID.
Do not use outside knowledge.
Do not guess, infer missing data, or fabricate numbers.
If the answer is not found in the retrieved content, reply: "I couldn't find relevant information in the
available files or my own knowledge."
Be concise, precise, and factual."""

examples = [
    {
        "input_query": "What do you know about IBM earnings in Q4, 2025? Summarize in one
sentence",
        "expected_answer": "IBM reported strong fourth-quarter results with revenue rising 12% to $19.7
billion, driven by double-digit growth in its Software and Infrastructure segments and a generative AI
book of business that has now surpassed $12.5 billion"
    },
    {
        "input_query": "What was the total value of IBM's generative AI book of business as reported in
the fourth quarter of 2025?",
        "expected_answer": "IBM reported that its generative AI book of business now stands at more
than $12.5 billion."
    },
    {
        "input_query": "What was IBM's reported free cash flow for the full year of 2025?",
        "expected_answer": (
            "IBM reported a full-year free cash flow of $14.7 billion, which was an increase of $2.0 billion
year-over-year"
        )
    },
    {
        "input_query": "How did the Software segment perform in terms of revenue during the fourth
quarter of 2025?",
        "expected_answer": (
            "The Software segment generated $9.0 billion in revenue, representing an increase of 14
```

```

percent (or 11 percent at constant currency)"
    )
    },
]

# Use the Responses API to create a results table comparing not using vs using
# the vector_store
table = Table(
    title="Answer Comparison (With vs Without Vector Store)",
    show_lines=True,
)

table.add_column("Question", style="cyan", no_wrap=False)
table.add_column("Expected Answer", style="magenta", no_wrap=False)
table.add_column("Answer (No Vector Store)", style="yellow", no_wrap=False)
table.add_column("Answer (With Vector Store)", style="green", no_wrap=False)

for example in examples:
    question = example["input_query"]
    expected_answer = example["expected_answer"]

    # Ask question without vector_store
    response_no_vs = client.responses.create(
        model=model_id,
        input=question,
        instructions=system_instructions,
    )
    answer_no_vs = response_no_vs.output_text.strip()

    # Ask question with vector_store
    response_vs = client.responses.create(
        model=model_id,
        input=question,
        instructions=system_instructions,
        tools=[
            {
                "type": "file_search",
                "vector_store_ids": [vector_store.id],
            }
        ],
    )
    answer_vs = response_vs.output_text.strip()

    table.add_row(
        question,
        expected_answer,
        answer_no_vs,
        answer_vs,
    )

# The table will take a while to be printed, as multiple queries to the responses API will be done
print(table)

```

4.2.1.9. Conversations API

- Endpoint: `/v1/conversations`.

- **Providers:** All agents and inference providers configured in OpenShift AI.
- **Support level:** Technology Preview.

The Conversations API enables multi-turn, context-aware chats by managing server-side conversation state. Instead of manually passing **previous_response_id** between Responses API calls, you can create a conversation that automatically accumulates message history across multiple turns. This simplifies building AI applications where each turn in the conversation can reference context from all previous turns.

The Conversations API provides the following operations:

- **Create a conversation: POST /v1/conversations** - Creates a new conversation container with optional metadata.
- **Retrieve a conversation: GET /v1/conversations/{id}** - Retrieves a conversation by ID.
- **Update a conversation: POST /v1/conversations/{id}** - Updates a conversation's metadata.
- **Delete a conversation: DELETE /v1/conversations/{id}** - Removes a conversation and its history.
- **Create conversation items: POST /v1/conversations/{id}/items** - Adds items to a conversation.
- **List conversation items: GET /v1/conversations/{id}/items** - Retrieves all messages stored in a conversation.
- **Retrieve a conversation item: GET /v1/conversations/{id}/items/{item_id}** - Retrieves a specific item.
- **Delete a conversation item: DELETE /v1/conversations/{id}/items/{item_id}** - Removes an item from a conversation.

To use a conversation with the Responses API, pass the **conversation** parameter instead of **previous_response_id** when calling **/v1/responses**.

Example code in Python:

```
model_id = "your-model-id"

# Step 1: Create a conversation
conversation = client.conversations.create(
    metadata={"topic": "pet-care", "user": "demo-user"}
)
conversation_id = conversation.id

# Step 2: Send messages using the Responses API with conversation_id
# Turn 1
response1 = client.responses.create(
    model=model_id,
    input="I have a rabbit. What is its living quarters called?",
    conversation=conversation_id,
    store=True, # Persist each response as a conversation item
    instructions="You are a helpful assistant. Keep responses brief.",
)
```

```

print(response1.output_text)

# Turn 2: The response can use context from Turn 1
response2 = client.responses.create(
    model=model_id,
    input="I also have a dog. What are its living quarters called?",
    conversation=conversation_id,
    store=True,
)
print(response2.output_text)

# Turn 3: The response can use context from previous turns
response3 = client.responses.create(
    model=model_id,
    input="List the living quarters I need for all my pets.",
    conversation=conversation_id,
    store=True,
)
print(response3.output_text)

# Step 3: List all messages in the conversation
items = client.conversations.items.list(conversation_id, order="asc")
for item in items.data:
    print(f"{item.role}: {item.content}")

# Step 4: Clean up
client.conversations.delete(conversation_id)

```



NOTE

The Conversations API is a Technology Preview feature in OpenShift AI. While functional and suitable for evaluation, some endpoints and parameters might change in future releases. This API is not recommended for production use.

Additional resources

- [OpenAI API compatibility in OGX](#)

4.2.2. OpenAI compatibility for RAG APIs in OGX

OpenShift AI supports OpenAI-compatible request and response schemas for OGX retrieval-augmented generation (RAG) workflows. This compatibility allows you to use OpenAI clients, tools, and schemas with OGX for managing files, vector stores, and executing RAG queries through the Responses API.

OpenAI compatibility enables the following capabilities:

- You can use OpenAI SDKs and tools with OGX by pointing the client to the OGX OpenAI-compatible API path.
- You can manage files and vector stores by using OpenAI-compatible endpoints and invoke RAG workflows by using the Responses API with the **file_search** tool.

When configuring clients, the required **base_url** depends on the SDK that you use:

- **OpenAI SDKs** When you use an OpenAI-compatible SDK (for example, the OpenAI Python client), you must include the **/v1** path suffix in the base URL. For example:

```
`http://ogx-service:8321/v1`
```

- **OGX SDK (`ogx_client`)** When you use the native OGX SDK, set the base URL to the OGX service endpoint without the **/v1** suffix. The SDK automatically appends the correct API paths. For example:

```
`http://ogx-service:8321`
```



IMPORTANT

When you use OpenAI-compatible SDKs or send raw HTTP requests to OGX, always include the **/v1** path suffix in the base URL.

Using the service endpoint without **/v1** results in request failures.

Additional resources

- [OpenAI API compatibility in OGX](#)
- [OpenAI API reference](#)
- [OGX Python client API](#)

4.3. OGX API PROVIDER SUPPORT

You can use OGX to enable various Provider APIs and providers in OpenShift AI. The following table lists the supported providers included in OpenShift AI, enablement environment variables, disconnected environment support, and its current support status.



WARNING

The support status of the OGX API providers has shifted between Technology Preview and Developer Preview across OpenShift AI versions.

Provider API	Providers	How to Enable	Disconnected support	Support status
Responses	<code>inline::builtin</code>	Enabled by default	Yes	Developer Preview
Messages	<code>inline::builtin</code>	Enabled by default	Yes	Developer Preview

Provider API	Providers	How to Enable	Disconnected support	Support status
Dataset_O	inline::localfs	Enabled by default	Yes	Technology Preview
	remote::huggingface	Enabled by default	No	Technology Preview
Files	inline::localfs	Enabled by default	No	Technology Preview
	remote::s3	Set the ENABLE_S3 environment variable to "true"	Yes	Developer Preview
Inference	remote::vllm	Set the VLLM_URL environment variable	Yes	Technology Preview
	inline::sentence-transformers	Set the ENABLE_SENTENCE_TRANSFORMERS environment variable to "true"	Yes	Technology Preview
	remote::azure	Set the AZURE_API_KEY environment variable	No	Technology Preview
	remote::gemini	Set the ENABLE_GEMINI environment variable	No	Developer Preview
	remote::anthropic	Set the ANTHROPIC_API_KEY environment variable	No	Developer Preview
	remote::bedrock	Set the AWS_ACCESS_KEY_ID environment variable	No	Technology Preview
	remote::openai	Set the OPENAI_API_KEY environment variable	No	Technology Preview
	remote::vertexai	Set the VERTEX_AI_PROJECT environment variable	No	Technology Preview

Provider API	Providers	How to Enable	Disconnected support	Support status
	remote::watsonx	Set the WATSONX_API_KEY environment variable	No	Technology Preview
Tool_Runtime	remote::model-context-protocol	Enabled by default	No	Developer Preview
	inline::rag-runtime	Enabled by default	No	Developer Preview
	remote::brave-search	Enabled by default	No	Developer Preview
	remote::tavily-search	Enabled by default	No	Developer Preview
Vector_IO	inline::faiss	Set the ENABLE_FAISS environment variable	No	Technology Preview
	inline::milvus	Set the ENABLE_INLINE_MILVUS environment variable to "true"	Yes	Technology Preview
	remote::milvus	Set the MILVUS_ENDPOINT environment variable	Yes	Technology Preview
	remote::pgvector	Set the ENABLE_PGVECTOR environment variable	Yes	Technology Preview
	remote::qdrant	Set the ENABLE_QDRANT environment variable	Yes	Technology Preview
File Processors	inline::auto	Enabled by default	No	Developer Preview
	inline::docling	Dependency only. Requires a custom config.yaml file	No	Developer Preview
	inline::markdown	Dependency only. Requires a custom config.yaml file	No	Developer Preview

Provider API	Providers	How to Enable	Disconnected support	Support status
	inline::pypdf	Dependency only. Requires a custom config.yaml file	No	Developer Preview
	remote::docling-serve	Dependency only. Requires a custom config.yaml file	No	Developer Preview



NOTE

Any providers labeled as **Dependency only** are not included in the default runtime **config.yaml** file, but their dependencies are pre-installed in the container image. To use those providers, pass a custom **config.yaml** at runtime that includes the provider definitions.

4.4. OPENAI-COMPATIBLE FILE CITATION ANNOTATIONS

OGX supports OpenAI-compatible file citation annotations in Responses API outputs when using the **file_search** tool. These annotations enable applications to trace generated responses back to source documents without requiring changes to existing OpenAI client code.

4.4.1. OpenAI-compatible file citation annotations in OGX

OpenShift AI provides OpenAI-compatible file citation annotations in Responses API outputs when using retrieval-augmented generation (RAG) with the **file_search** tool. These annotations enable applications to trace generated responses back to the source files used during retrieval without requiring changes to existing OpenAI client code. When you use the Responses API with the **file_search** tool, OGX returns citation metadata that references the source file used to generate the response. Annotations are enabled by default.

Citation annotations have the following characteristics:

- They follow the same response structure defined by OpenAI.
- They appear in the **annotations** field of **output_text** response content.
- They identify the source file by ID and filename.
- They provide document-level attribution.

This feature improves transparency for RAG workflows while maintaining schema compatibility with OpenAI request and response formats.

In OpenShift AI, the following annotation capabilities are supported:

- Annotations are returned only through the Responses API.
- Annotations are returned only when using the **file_search** tool.

- The **file_citation** annotation type is supported.
- Attribution is provided at the document level.

Additional resources

- [OpenAI compatibility for RAG APIs in OGX](#)
- [OpenAI-compatible APIs in OGX](#)

4.4.2. Viewing file citation annotations in Responses API output

When you query ingested content by using the **file_search** tool with the Responses API, OGX returns OpenAI-compatible **file_citation** annotations. These annotations identify the source files used during retrieval.

Prerequisites

- You have deployed a OGX server.
- You have configured at least one inference model.
- You have created a vector store and ingested content into it.
- You can successfully execute a RAG query by using the **file_search** tool, as described in [Querying ingested content in a Llama model](#).
- You have access to a client environment, such as a Jupyter notebook or an OpenAI SDK client, that is correctly configured to send authenticated requests to the OGX server.



NOTE

This procedure requires that content has already been ingested into a vector store. If no content is available, RAG queries return empty or non-contextual responses.

Procedure

1. In a Jupyter notebook cell or other configured client environment, run a RAG query by using the **file_search** tool.

```
response = client.responses.create(
    model=model_id,
    input=query,
    instructions=system_instructions,
    tools=[
        {
            "type": "file_search",
            "vector_store_ids": [vector_store_id],
        }
    ],
)
```

2. Inspect the full response object rather than only the **output_text** property.

```
response.output
```

3. Access the **annotations** array.

```
annotations = response.output[0].content[0].annotations
print(annotations)
```

4. Review the **file_citation** annotation fields.

Example output:

```
[
  {
    "type": "file_citation",
    "file_id": "file-57610eaac6364459bfefae60377837b7",
    "filename": "redbankfinancial_about.pdf",
    "index": 139
  }
]
```

Each **file_citation** annotation includes the following fields:

- **file_id**: The identifier of the retrieved file.
- **filename**: The name of the source file.
- **index**: The index of the cited file in the list of files.

Multiple annotations can reference the same index position.

Optional: Using the OpenAI-compatible HTTP endpoint

If you use raw HTTP requests or an OpenAI SDK, send requests to the following endpoint:

/v1/responses

Ensure that your base URL includes the **/v1** path suffix, as described in [OpenAI compatibility for RAG APIs in OGX](#).



NOTE

The accuracy and consistency of citation annotations depend on the capabilities of the underlying language model. Smaller or less capable models might produce less precise attributions, even when retrieval is functioning correctly. If citation results are incomplete or inconsistent, verify the model configuration and consider using a larger or more capable model.

Optional: Using the OpenAI-compatible endpoint

When you use an OpenAI SDK, configure the client **base_url** to include the **/v1** path suffix. The SDK automatically appends the appropriate endpoint path, such as **/responses**.

For example:

http://ogx-service:8321/v1

When you send raw HTTP requests, include both the **/v1** path suffix and the **/responses** endpoint in the full request URL.

For example:

http://ogx-service:8321/v1/responses

Ensure that **/v1** is included only once in the base URL. Do not append **/v1** multiple times.

For more information, see [OpenAI compatibility for RAG APIs in OGX](#).



NOTE

The accuracy and consistency of citation annotations depend on the capabilities of the underlying language model. Smaller or less capable models might produce less precise attributions, even when retrieval is functioning correctly. If citation results are incomplete or inconsistent, verify the model configuration and consider using a larger or more capable model.

Verification

- The response includes an **annotations** array under **output[].content[]**.
- Each annotation has **"type": "file_citation"**.
- The **file_id** and **filename** correspond to files stored in the specified vector store.

4.4.3. File citation annotation reference

This reference describes the **file_citation** annotation type returned by OGX through the OpenAI-compatible Responses API.

4.4.3.1. Annotation location

Annotations are returned in the **annotations** field of **output_text** content items within the **output[].content[]** structure of the Responses API response.

```
"output": [
  {
    "content": [
      {
        "type": "output_text",
        "text": "Example generated response.",
        "annotations": [ ... ]
      }
    ]
  }
]
```

4.4.3.2. Supported annotation type

In OpenShift AI, OGX returns the **file_citation** annotation type when using the **file_search** tool.

URL citation annotations

The **url_citation** type is defined in the OpenAI schema but is not produced by OGX in OpenShift AI 3.3.

4.4.3.3. File citation fields

The **file_citation** annotation includes the following fields:

Field	Type	Description
type	string	Always file_citation
file_id	string	Identifier of the source file used during retrieval
filename	string	Name of the source file
index	integer	Index of the cited file in the list of files.

4.4.3.4. Annotation behavior

- Attribution is provided at the document level.
- Multiple annotations can reference the same index position.
- Chunk-level and token-level attribution are not supported.
- Annotations follow the OpenAI response schema without modification.

CHAPTER 5. ACTIVATING THE OGX OPERATOR

You can activate the OGX Operator on your OpenShift cluster by setting its **managementState** to **Managed** in the OpenShift AI Operator **DataScienceCluster** custom resource (CR). This setting enables Llama-based model serving without reinstalling or directly editing Operator subscriptions. You can edit the CR in the OpenShift web console or by using the OpenShift CLI (**oc**).



NOTE

As an alternative to following the steps in this procedure, you can activate the OGX Operator from the OpenShift CLI (**oc**) by running the following command:

```
$ oc patch datasciencecluster <name> --type=merge -p {"spec":{"components":{"ogx":{"managementState":"Managed"}}}}
```

Replace **<name>** with your **DataScienceCluster** name, for example, **default-dsc**.

Prerequisites

- You have installed OpenShift 4.19 or newer.
- You have cluster administrator privileges.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS
- You have installed the Red Hat OpenShift AI Operator on your cluster.
- You have a **DataScienceCluster** custom resource in your environment; the default is **default-dsc**.
- Your infrastructure supports GPU-enabled instance types, for example, **g4dn.xlarge** on AWS.
- You have enabled GPU support in OpenShift AI, including installing the Node Feature Discovery Operator and NVIDIA GPU Operator. For more information, see [Installing the Node Feature Discovery Operator](#) and [Enabling NVIDIA GPUs](#).
- You have created a **NodeFeatureDiscovery** resource instance on your cluster, as described in [Installing the Node Feature Discovery Operator and creating a NodeFeatureDiscovery instance](#) in the NVIDIA documentation.
- You have created a **ClusterPolicy** resource instance with default values on your cluster, as described in [Creating the ClusterPolicy instance](#) in the NVIDIA documentation.



NOTE

For IBM Power, **ppc64le** architectures, CPU-only deployments are fully supported.

Procedure

1. Log in to the OpenShift web console as a cluster administrator.
2. Go to the **Installed Operators** page. The navigation path depends on your OpenShift version:
 - On OpenShift 4.20 and later, click **Ecosystem → Installed Operators**.
 - On OpenShift 4.19, click **Operators → Installed Operators**.
3. Click the **Red Hat OpenShift AI Operator** to open its details.
4. Click the **Data Science Cluster** tab.
5. On the **DataScienceClusters** page, click the **default-dsc** object.
6. Click the **YAML** tab.
An embedded YAML editor opens, displaying the configuration for the **DataScienceCluster** custom resource.
7. In the YAML editor, locate the **spec.components** section. If the **ogx** field does not exist, add it. Then, set the **managementState** field to **Managed**:

```
spec:
  components:
    ogx:
      managementState: Managed
```

8. Click **Save** to apply your changes.

Verification

After you activate the OGX Operator, verify that it is running in your cluster:

1. In the OpenShift web console, click **Workloads → Pods**.
2. From the **Project** list, select the **redhat-ods-applications** namespace.
3. Confirm that a pod with the label **name=ogx-k8s-operator** is displayed and has a status of **Running**.

CHAPTER 6. DEPLOYING A OGX SERVER

OGX allows you to create and deploy a server that enables various APIs for accessing AI services in your OpenShift AI cluster. You can create a **OGXServer** custom resource for your desired use cases. You are responsible for provisioning and managing the PostgreSQL instance. The PostgreSQL database can be deployed in-cluster or hosted externally, as long as it is reachable from the cluster network.

The included procedure provides an example **OGXServer** CR that deploys a OGX server that enables the following setup:

- A connection to a vLLM inference service with a **llama32-3b** model.
- A connection to a remote vector database.
- Allocated persistent storage.
- Orchestration endpoints.

Prerequisites

- You have installed OpenShift 4.19 or newer.
- You have logged in to Red Hat OpenShift AI.
- You have cluster administrator privileges for your OpenShift cluster.
- You have activated the OGX Operator in your cluster.
- You have access to a PostgreSQL version 14 or later instance that is reachable from the OpenShift cluster network.
- You have PostgreSQL credentials for that instance that allow OGX to create the database and tables.
- You know the PostgreSQL hostname and database port to use for the **POSTGRES_HOST** and **POSTGRES_PORT** environment variables.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS

Procedure

1. In the OpenShift web console, select **Administrator** → **Quick Create** () → **Import YAML**, and create a CR similar to the following example **ogx-custom-server.yaml** file:

Example ogx-custom-server.yaml

```
apiVersion: ogx.io/v1beta1
kind: OGXServer
metadata:
  name: ogx-custom-server
```

```

namespace: <project-name> # Replace with your OpenShift project
spec:
  distribution:
    name: rh-dev
  workload:
    replicas: 1
    overrides:
      env:
        - name: VLLM_URL
          value: 'https://llama32-3b.ogx.svc.cluster.local/v1'
        - name: INFERENCE_MODEL
          value: llama32-3b
        - name: VLLM_TLS_VERIFY
          value: 'false'
        - name: POSTGRES_HOST
          value: <postgres-host>
        - name: POSTGRES_PORT
          value: '<postgres-port>' # Default PostgreSQL port is 5432
        - name: POSTGRES_DB
          value: ogx
        - name: POSTGRES_USER
          value: ogx
        - name: POSTGRES_PASSWORD
          valueFrom:
            secretKeyRef:
              key: password
              name: postgres-secret ❶
      name: ogx
      port: 8321
  distribution:
    name: 'rh-dev'
  storage:
    size: 20Gi
    mountPath: <custom-mount-path> ## Defaults to /opt/app-root/src/.ogx/distributions/rh/

```

- ❶ Create the secret in the same namespace as the **OGXServer** resource. Avoid placing passwords directly on the command line, as they can be stored in shell history. Instead, create a file that contains only the database password and use that file to create the secret, or create the secret by using the OpenShift web console.

For example:

```

$ oc create secret generic postgres-secret --from-file=password=pg-password.txt -n
<project-name>
$ rm -f pg-password.txt

```

For more information about creating and managing Secrets, see [Providing sensitive data to pods by using secrets](#).

Ensure that the file **pg-password.txt** contains only the database password and is deleted after the secret is created.

OGX automatically creates the metadata database specified by the **POSTGRES_DB** environment variable if it does not already exist, provided that the PostgreSQL user has sufficient privileges.

Verification

1. Check that the custom resource was created with the following command:

```
$ oc get ogxserver -n ogx
```

2. Check the running pods with the following command:

```
$ oc get pods -n ogx | grep ogx-custom-server
```

3. Check the logs with the following command:

```
$ oc logs -n ogx -l app=ogx
```

Example output

```
INFO: Started server process
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://[':::', '0.0.0.0']:8321
```

CHAPTER 7. TESTING YOUR VLLM MODEL ENDPOINTS

To verify that your deployed Llama 3.2 model is accessible externally, ensure that your vLLM model server is exposed as a network endpoint. You can then test access to the model from outside both the OpenShift cluster and the OpenShift AI interface.



IMPORTANT

If you selected **Make deployed models available through an external route** during deployment, your vLLM model endpoint is already accessible outside the cluster. You do not need to manually expose the model server. Manually exposing vLLM model endpoints, for example, by using **oc expose**, creates an unsecured route unless you configure authentication. Avoid exposing endpoints without security controls to prevent unauthorized access.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have logged in to Red Hat OpenShift AI.
- You have activated the OGX Operator in OpenShift AI.
- You have deployed an inference model, for example, the llama-3.2-3b-instruct model.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS

Procedure

1. Open a new terminal window.
 - a. Log in to your OpenShift cluster from the CLI:
 - b. In the upper-right corner of the OpenShift web console, click your user name and select **Copy login command**.
 - c. After you have logged in, click **Display token**.
 - d. Copy the **Log in with this token** command and paste it in the OpenShift CLI (**oc**).

```
$ oc login --token=<token> --server=<openshift_cluster_url>
```

2. If you enabled **Require token authentication** during model deployment, retrieve your token:

```
$ export MODEL_TOKEN=$(oc get secret default-name-llama-32-3b-instruct-sa -n <project name> --template={{ .data.token }} | base64 -d)
```

3. Obtain your model endpoint URL:

- If you enabled **Make deployed models available through an external route** during model deployment, click **Endpoint details** on the **Deployments** page in the OpenShift AI dashboard to obtain your model endpoint URL.
- In addition, if you did not enable **Require token authentication** during model deployment, you can also enter the following command to retrieve the endpoint URL:

```
$ export MODEL_ENDPOINT="https://$(oc get route llama-32-3b-instruct -n <project name> --template={{ .spec.host }})"
```

4. Test the endpoint with a sample chat completion request:

- If you did not enable **Require token authentication** during model deployment, enter a chat completion request. For example:

```
$ curl -X POST $MODEL_ENDPOINT/v1/chat/completions \
-H "Content-Type: application/json" \
-d '{
  "model": "llama-32-3b-instruct",
  "messages": [
    {
      "role": "user",
      "content": "Hello"
    }
  ]
}'
```

- If you enabled **Require token authentication** during model deployment, include a token in your request. For example:

```
curl -s -k $MODEL_ENDPOINT/v1/chat/completions \
--header "Authorization: Bearer $MODEL_TOKEN" \
--header 'Content-Type: application/json' \
-d '{
  "model": "llama-32-3b-instruct",
  "messages": [
    {
      "role": "user",
      "content": "can you tell me a funny joke?"
    }
  ]
}' | jq .
```



NOTE

The **-k** flag disables SSL verification and should only be used in test environments or with self-signed certificates.

Verification

Confirm that you received a JSON response containing a chat completion. For example:

```
{
  "id": "chatcmpl-05d24b91b08a4b78b0e084d4cc91dd7e",
  "object": "chat.completion",
```

```
"created": 1747279170,  
"model": "llama-32-3b-instruct",  
"choices": [{  
  "index": 0,  
  "message": {  
    "role": "assistant",  
    "reasoning_content": null,  
    "content": "Hello! It's nice to meet you. Is there something I can help you with or would you like to  
chat?",  
    "tool_calls": []  
  },  
  "logprobs": null,  
  "finish_reason": "stop",  
  "stop_reason": null  
}],  
"usage": {  
  "prompt_tokens": 37,  
  "total_tokens": 62,  
  "completion_tokens": 25,  
  "prompt_tokens_details": null  
},  
"prompt_logprobs": null  
}
```

If you do not receive a response similar to the example, verify that the endpoint URL and token are correct, and ensure your model deployment is running.

CHAPTER 8. CONFIGURE AWS BEDROCK AS AN OGX INFERENCE PROVIDER

OGX now supports standard AWS Signature Version 4 (SigV4) signing for Amazon Bedrock requests. Using the **remote::bedrock** provider, OGX can use your existing AWS credentials and eliminates custom authentication plumbing and manual token management.

Key Benefits

- **Native AWS identity support:** Functions with IAM roles, IRSA, STS web identity, AWS profiles, and short-lived credentials.
- **Minimal application changes:** Maintain your existing OpenAI-compatible client while OGX utilizes provider-specific authentication.
- **Simplified operations:** Eliminates bearer token management workflows at the application layer.
- **Backward compatible:** Preserves existing bearer token setups without breaking current implementations.

This procedure configures and runs OGX as a local standalone server that proxies inference requests to AWS Bedrock.

Prerequisites

- You have the OGX server CLI installed on your local machine.
- You have **curl** and **jq** installed on your local machine.
- You have the **openai** Python package installed on your local machine.
- You have an AWS account with Amazon Bedrock enabled.
 - Your IAM role or user has permission to invoke the target Bedrock model, for example **bedrock:InvokeModel**.
 - If you use STS role assumption or IRSA, the trust relationship for the IAM role is configured to allow your identity to assume it.
 - The Bedrock model you plan to use is available in your selected AWS region.

Procedure

1. Export your AWS region with the following command:

```
$ export AWS_DEFAULT_REGION=us-west-2
```

2. Export the credentials for your environment. Choose the option that matches your AWS credential configuration:

AWS CLI profile

Use this option if you manage credentials with named AWS CLI profiles.

```
$ export AWS_PROFILE=default
```

Replace **default** with your profile name.

STS role assumption

Use this option if your environment requires cross-account or scoped role access.

```
$ export AWS_ROLE_ARN=arn:aws:iam::<account_id>:role/<role_name>
```

Your current credentials, such as an AWS CLI profile or environment variables, must have permission to assume this role.

Web identity (IRSA)

Use this option when running OGX inside an EKS pod configured with IAM Roles for Service Accounts. The token file is mounted automatically by the EKS pod identity webhook.

```
$ export AWS_ROLE_ARN=arn:aws:iam::<account_id>:role/<role_name>
$ export
AWS_WEB_IDENTITY_TOKEN_FILE=/var/run/secrets/eks.amazonaws.com/serviceaccount/token
```

The token file path must match the mount configured in your pod's service account.

3. Create a **config.yaml** file that enables the **remote::bedrock** provider, includes your AWS credentials, and storage configurations.

Example AWS Bedrock YAML configuration

```
version: 2
distro_name: bedrock-sigv4-demo
apis:
  - inference
  - models
providers:
  inference:
    - provider_id: bedrock-inference
      provider_type: remote::bedrock
      config:
        # aws_bedrock_bearer_token intentionally omitted so OGX uses the AWS credential
        chain
        region_name: ${env.AWS_DEFAULT_REGION:=us-west-2}
        aws_role_arn: ${env.AWS_ROLE_ARN:=}
        aws_web_identity_token_file: ${env.AWS_WEB_IDENTITY_TOKEN_FILE:=}
        aws_role_session_name: ${env.AWS_ROLE_SESSION_NAME:=ogx-bedrock-demo}
        session_ttl: ${env.AWS_SESSION_TTL:=3600}

storage:
  backends:
    kv_default:
      type: kv_sqlite
      db_path: ../ogx/kvstore.db
    sql_default:
      type: sql_sqlite
      db_path: ../ogx/sql_store.db
  stores:
```

```

metadata:
  namespace: registry
  backend: kv_default
inference:
  table_name: inference_store
  backend: sql_default
  max_write_queue_size: 10000
  num_writers: 4
prompts:
  namespace: prompts
  backend: kv_default

registered_resources:
  models:
    - metadata: {}
      model_id: openai.gpt-oss-20b-1:0
      provider_id: bedrock-inference
      provider_model_id: openai.gpt-oss-20b-1:0
      model_type: llm

```

4. Start the OGX Server with the following command:

```
$ ogx run --port 8321 ./config.yaml
```

5. Confirm the model is registered:

```
$ MODEL_ID=$(curl -s http://localhost:8321/v1/models | jq -r '.data[0].id // empty')
test -n "$MODEL_ID" && echo "Using model: $MODEL_ID"
```

Verification

1. Check the request with a **curl** command:

```
$ curl -s -X POST "http://localhost:8321/v1/chat/completions" \
-H "Content-Type: application/json" \
-d "{
  \"model\": \"$MODEL_ID\",
  \"messages\": [{\"role\": \"user\", \"content\": \"Which planet do humans live on?\"}],
  \"stream\": false
}" | jq -r '.choices[0].message.content'
```

2. Check the request with the OpenAI Python client:

```

from openai import OpenAI

client = OpenAI(
    base_url="http://localhost:8321/v1",
    api_key="ogx",
)

response = client.chat.completions.create(
    model="<model_id>",
    messages=[{"role": "user", "content": "Which planet do humans live on?"}],
    stream=False,

```

```
)  
    print(response.choices[0].message.content)
```

where:

<model_id>

Specifies the model ID returned by the **/v1/models** endpoint.

CHAPTER 9. DEPLOYING A LLAMA MODEL WITH KSERVE

To use OGX and retrieval-augmented generation (RAG) workloads in OpenShift AI, you must deploy a Llama model with a vLLM model server and configure KServe in KServe RawDeployment mode.



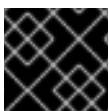
IMPORTANT

When deploying models using **KServe on IBM Power (ppc64le)**, ensure that you use only supported parameters for the model configuration.

Half (FP16) precision is not currently supported on this architecture. Attempting to use FP16 may result in a **NotImplementedError: "rotary_embedding_impl" not implemented for 'Half'** error.

Prerequisites

- You have installed OpenShift 4.19 or newer.
- You have logged in to Red Hat OpenShift AI.
- You have cluster administrator privileges for your OpenShift cluster.
- You have activated the OGX Operator.
- You have installed KServe.
- You have enabled the model serving platform. For more information about enabling the model serving platform, see [Enabling the model serving platform](#).
- You can access the model serving platform in the dashboard configuration. For more information about setting dashboard configuration options, see [Customizing the dashboard](#).
- You have enabled GPU support in OpenShift AI, including installing the Node Feature Discovery Operator and NVIDIA GPU Operator. For more information, see [Installing the Node Feature Discovery Operator](#) and [Enabling NVIDIA GPUs](#).
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS
- You have created a project.
- The vLLM serving runtime is installed and available in your environment.
- You have created a storage connection for your model that contains a **URI - v1** connection type. This storage connection must define the location of your Llama 3.2 model artifacts. For example, **oci://quay.io/redhat-ai-services/modelcatalog:llama-3.2-3b-instruct**. For more information about creating storage connections, see [Adding a connection to your project](#).



PROCEDURE

These steps are only supported in OpenShift AI versions 2.19 and later.

1. In the OpenShift AI dashboard, navigate to the project details page and click the **Deployments** tab.
2. In the **Model serving platform** tile, click **Select model**.
3. Click the **Deploy model** button.
The **Deploy model** dialog opens.
4. Configure the deployment properties for your model:
 - a. In the **Model deployment name** field, enter a unique name for your deployment.
 - b. In the **Serving runtime** field, select **vLLM NVIDIA GPU serving runtime for KServe** from the drop-down list.
 - c. In the **Deployment mode** field, select **KServe RawDeployment** from the drop-down list.
 - d. Set **Number of model server replicas to deploy** to **1**.
 - e. In the **Model server size** field, select **Custom** from the drop-down list.
 - Set **CPUs requested** to **1 core**.
 - Set **Memory requested** to **10 GiB**.
 - Set **CPU limit** to **2 core**.
 - Set **Memory limit** to **14 GiB**.
 - Set **Accelerator** to **NVIDIA GPUs**.
 - Set **Accelerator count** to **1**.
 - f. From the **Connection type**, select a relevant data connection from the drop-down list.
5. In the **Additional serving runtime arguments** field, specify the following recommended arguments:

```
--dtype=half
--max-model-len=20000
--gpu-memory-utilization=0.95
--enable-chunked-prefill
--enable-auto-tool-choice
--tool-call-parser=llama3_json
--chat-template=/app/data/template/tool_chat_template_llama3.2_json.jinja
```

- a. Click **Deploy**.



NOTE

Model deployment can take several minutes, especially for the first model that is deployed on the cluster. Initial deployment may take more than 10 minutes while the relevant images download.

Verification

1. Verify that the **kserve-controller-manager** and **odh-model-controller** pods are running:
 - a. Open a new terminal window.
 - b. Log in to your OpenShift cluster from the CLI:
 - c. In the upper-right corner of the OpenShift web console, click your user name and select **Copy login command**.
 - d. After you have logged in, click **Display token**.
 - e. Copy the **Log in with this token** command and paste it in the OpenShift CLI (**oc**).

```
$ oc login --token=<token> --server=<openshift_cluster_url>
```

- f. Enter the following command to verify that the **kserve-controller-manager** and **odh-model-controller** pods are running:

```
$ oc get pods -n redhat-ods-applications | grep -E 'kserve-controller-manager|odh-model-controller'
```

- g. Confirm that you see output similar to the following example:

```
kserve-controller-manager-7c865c9c9f-xyz12 1/1 Running 0 4m21s
odh-model-controller-7b7d5fd9cc-wxy34 1/1 Running 0 3m55s
```

- h. If you do not see either of the **kserve-controller-manager** and **odh-model-controller** pods, there could be a problem with your deployment. In addition, if the pods appear in the list, but their **Status** is not set to **Running**, check the pod logs for errors:

```
$ oc logs <pod-name> -n redhat-ods-applications
```

- i. Check the status of the inference service:

```
$ oc get inferencesservice -n ogx
$ oc get pods -n <project name> | grep llama
```

- The deployment automatically creates the following resources:
 - A **ServingRuntime** resource.
 - An **InferenceService** resource, a **Deployment**, a pod, and a service pointing to the pod.
- Verify that the server is running. For example:

```
$ oc logs llama-32-3b-instruct-predictor-77f6574f76-8nl4r -n <project name>
```

Check for output similar to the following example log:

```
INFO 2025-05-15 11:23:52,750 __main__:498 server: Listening on ['::',
'0.0.0.0']:8321
INFO: Started server process [1]
INFO: Waiting for application startup.
```

```

INFO    2025-05-15 11:23:52,765 __main__:151 server: Starting up
INFO:   Application startup complete.
INFO:   Uvicorn running on http://['::', '0.0.0.0']:8321 (Press CTRL+C to quit)

```

- The deployed model displays in the **Deployments** tab on the project details page for the project it was deployed under.
2. If you see a **ConvertTritonGPToLLVM** error in the pod logs when querying the **/v1/chat/completions** API, and the vLLM server restarts or returns a **500 Internal Server** error, apply the following workaround:
- Before deploying the model, remove the **--enable-chunked-prefill** argument from the **Additional serving runtime arguments** field in the deployment dialog.

The error is displayed similar to the following:

```

/opt/vllm/lib64/python3.12/site-packages/vllm/attention/ops/prefix_prefill.py:36:0: error:
Failures have been detected while processing an MLIR pass pipeline
/opt/vllm/lib64/python3.12/site-packages/vllm/attention/ops/prefix_prefill.py:36:0: note:
Pipeline failed while executing ['`ConvertTritonGPToLLVM`' on 'builtin.module' operation]:
reproducer generated at `std::errs, please share the reproducer above with Triton project.`
INFO:   10.129.2.8:0 - "POST /v1/chat/completions HTTP/1.1" 500 Internal Server Error

```

CHAPTER 10. SELECT AND DEPLOY A VECTOR DATABASE

When your application requires retrieval-augmented generation (RAG), choose and configure a vector store so that you can store and query document embeddings for retrieval during inference. OGX in OpenShift AI supports remote Milvus, PostgreSQL with the pgvector extension, and Qdrant as vector store providers. After configuring a vector store, you can build a RAG application, as described in [Deploying a RAG stack in a project](#).

10.1. OVERVIEW OF VECTOR DATABASES

Vector databases are a core component of retrieval-augmented generation (RAG) in OpenShift AI. They store and index vector embeddings that represent the semantic meaning of text or other data. When integrated with OGX, vector databases enable applications to retrieve relevant context and combine it with large language model (LLM) inference.

Vector databases provide the following capabilities:

- Store vector embeddings generated by embedding models.
- Support efficient similarity search to retrieve semantically related content.
- Enable RAG workflows by supplying the LLM with contextually relevant data.

In OpenShift AI, vector databases are configured and managed through the OGX Operator as part of a **OGXServer**. PostgreSQL is the default and recommended metadata store for OGX, supporting production-ready persistence, concurrency, and scalability.

The following vector database options are supported in OpenShift AI:

- **Remote Milvus** Remote Milvus runs as a standalone vector database service, either within the cluster or as an external managed deployment. This option is suitable for large-scale or production-grade RAG workloads that require high availability, horizontal scalability, and isolation from the OGX server. In OpenShift environments, Milvus typically requires an accompanying etcd service for coordination. For more information, see [Providing redundancy with etcd](#).
- **Remote PostgreSQL with pgvector** PostgreSQL with the pgvector extension provides a production-ready vector database option that integrates vector similarity search directly into PostgreSQL. This option is well suited for environments that already operate PostgreSQL and require durable storage, transactional consistency, and centralized management. pgvector enables OGX to store embeddings and perform similarity search without deploying a separate vector database service.

Consider the following guidance when choosing a vector database for your RAG workloads:

- Use **Remote Milvus** when you require large-scale vector indexing and high-throughput similarity search.
- Use **PostgreSQL with pgvector** when you want production-ready persistence and integration with existing PostgreSQL-based data platforms.

SQLite-based storage is no longer recommended for production deployments. PostgreSQL-based backends provide improved reliability, concurrency, and scalability as OGX moves toward general availability.

10.1.1. Overview of Milvus vector databases

Milvus is an open source vector database designed for high-performance similarity search across large volumes of embedding data. In OpenShift AI, Milvus is supported as a vector store provider for OGX and enables retrieval-augmented generation (RAG) workloads that require efficient vector indexing, scalable search, and durable storage.

Production-grade OGX deployments default to PostgreSQL for metadata persistence. When Milvus is used as the vector store, PostgreSQL is typically used for OGX metadata, while Milvus manages vector indexes and similarity search.

Milvus vector databases provide the following capabilities in OpenShift AI:

- High-performance similarity search using Approximate Nearest Neighbor (ANN) algorithms
- Efficient indexing and query optimization for dense embeddings
- Persistent storage of vector data
- Integration with OGX through an OpenAI-compatible Vector Stores API

In a typical RAG workflow in OpenShift AI, the following responsibilities are separated:

- **Embedding generation** Embeddings are generated by the configured embedding provider. Remote embedding models are the recommended and default option for production deployments.
- **Vector storage and retrieval** Milvus stores embedding vectors and performs similarity search operations.
- **Metadata persistence** OGX stores vector store metadata, file references, and configuration state using PostgreSQL in production deployments.
- **OGX server** Coordinates ingestion, retrieval, and model inference through a unified API surface.

In OpenShift AI, Milvus can be used in the following operational modes:

- **Remote Milvus** Runs as a standalone service within your OpenShift project or as an external managed Milvus deployment. Remote Milvus is recommended for production-grade RAG workloads.

A remote Milvus deployment typically includes the following components:

- A **Milvus service** that exposes a gRPC endpoint (port 19530) for client traffic
- An **etcd service** that Milvus uses for metadata coordination, collection state, and index management
- **Persistent storage** for durable vector data

Milvus requires a dedicated etcd instance for metadata coordination, even when running in standalone mode. Do not use the OpenShift control plane etcd for this purpose. For more information about etcd, see [Providing redundancy with etcd](#).

**IMPORTANT**

You must deploy a dedicated etcd service for Milvus or connect Milvus to an external etcd instance. Do not share the OpenShift control plane etcd with application workloads.

Use **Remote Milvus** when you require scalable vector search, high-performance retrieval, and integration with production-grade OGX deployments in OpenShift AI.

For instructions on deploying Milvus as a remote vector database, see [Deploying a remote Milvus vector database](#).

10.1.2. Overview of pgvector vector databases

pgvector is an open source PostgreSQL extension that enables vector similarity search on embedding data stored in relational tables. In OpenShift AI, PostgreSQL with the pgvector extension is supported as a remote vector database provider for the OGX Operator. pgvector supports retrieval augmented generation workflows that require persistent vector storage while integrating with existing PostgreSQL environments.

pgvector vector databases provide the following capabilities in OpenShift AI:

- Storage of vector embeddings in PostgreSQL tables.
- Similarity search across embeddings by using pgvector distance metrics.
- Persistent storage of vectors alongside structured relational data.
- Integration with existing PostgreSQL security and operational tooling.

In a typical retrieval augmented generation workflow in OpenShift AI, your application uses the following components:

- **Inference provider** Generates embeddings and model responses.
- **Vector store provider** Stores embeddings and performs similarity search. When you use pgvector, PostgreSQL provides this capability as a remote vector store.
- **File storage provider** Stores the source files that are ingested into vector stores.
- **OGX server** Provides a unified API surface, including an OpenAI compatible Vector Stores API.

When you ingest content, OGX splits source material into chunks, generates embeddings, and stores them in PostgreSQL through the pgvector extension. When you query a vector store, OGX performs similarity search and returns the most relevant chunks for use in prompts.

In OpenShift AI, pgvector is used in the following operational mode:

- **Remote PostgreSQL with pgvector**, which runs as a standalone PostgreSQL database service accessed by the OGX server. This mode is suitable for development and production workloads that require persistent storage and integration with existing PostgreSQL infrastructure.

When you deploy PostgreSQL with the pgvector extension, you typically manage the following components:

- **Secrets** for PostgreSQL connection credentials.
- **Persistent storage** for durable database data.

- A **PostgreSQL service** that exposes a network endpoint.

PostgreSQL with pgvector does not require an external coordination service. Vector data, indexes, and metadata are stored directly in PostgreSQL tables and managed through standard database mechanisms.

Use PostgreSQL with pgvector when you require persistent vector storage and want to integrate vector search into existing PostgreSQL based data platforms within OpenShift AI. [Deploying a PostgreSQL instance with pgvector](#).

10.2. DEPLOYING A REMOTE MILVUS VECTOR DATABASE

To use Milvus as a remote vector database provider for OGX in OpenShift AI, you must deploy Milvus and its required etcd service in your OpenShift project. This procedure shows how to deploy Milvus in standalone mode without the Milvus Operator.



NOTE

The following example configuration is intended for testing or evaluation environments. For production-grade deployments, see <https://milvus.io/docs> in the Milvus documentation.

Prerequisites

- You have installed OpenShift 4.19 or newer.
- You have enabled GPU support in OpenShift AI. This includes installing the Node Feature Discovery operator and NVIDIA GPU Operators. For more information, see [Installing the Node Feature Discovery operator](#) and [Enabling NVIDIA GPUs](#).
- You have cluster administrator privileges for your OpenShift cluster.
- You are logged in to Red Hat OpenShift AI.
- You have a StorageClass available that can provision persistent volumes.
- You created a root password to secure your Milvus service.
- You have deployed an inference model with vLLM, for example, the llama-3.2-3b-instruct model, and you have selected **Make deployed models available through an external route** and **Require token authentication** during model deployment.
- You have the correct inference model identifier, for example, llama-3-2-3b.
- You have the model endpoint URL, ending with **/v1**, such as **https://llama-32-3b-instruct-predictor:8443/v1**.
- You have the API token required to access the model endpoint.
- You have installed the OpenShift command line interface (**oc**) as described in [Installing the OpenShift CLI](#).

Procedure



1. In the OpenShift console, click the **Quick Create** () icon and then click the **Import YAML** option.
2. Verify that your project is the selected project.
3. In the **Import YAML** editor, paste the following manifest and click **Create**:

```

apiVersion: v1
kind: Secret
metadata:
  name: milvus-secret
type: Opaque
stringData:
  root-password: "MyStr0ngP@ssw0rd"
---
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: milvus-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 20Gi
  volumeMode: Filesystem
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: etcd-deployment
labels:
  app: etcd
spec:
  replicas: 1
  selector:
    matchLabels:
      app: etcd
  strategy:
    type: Recreate
  template:
    metadata:
      labels:
        app: etcd
    spec:
      containers:
        - name: etcd
          image: quay.io/coreos/etcd:v3.5.5
          command:
            - etcd
            - --advertise-client-urls=http://127.0.0.1:2379
            - --listen-client-urls=http://0.0.0.0:2379
            - --data-dir=/etcd
          ports:
            - containerPort: 2379

```

```

    volumeMounts:
      - name: etcd-data
        mountPath: /etcd
    env:
      - name: ETCD_AUTO_COMPACTION_MODE
        value: revision
      - name: ETCD_AUTO_COMPACTION_RETENTION
        value: "1000"
      - name: ETCD_QUOTA_BACKEND_BYTES
        value: "4294967296"
      - name: ETCD_SNAPSHOT_COUNT
        value: "50000"
    volumes:
      - name: etcd-data
        emptyDir: {}
    restartPolicy: Always
---
apiVersion: v1
kind: Service
metadata:
  name: etcd-service
spec:
  ports:
    - port: 2379
      targetPort: 2379
  selector:
    app: etcd
---
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    app: milvus-standalone
  name: milvus-standalone
spec:
  replicas: 1
  selector:
    matchLabels:
      app: milvus-standalone
  strategy:
    type: Recreate
  template:
    metadata:
      labels:
        app: milvus-standalone
    spec:
      containers:
        - name: milvus-standalone
          image: milvusdb/milvus:v2.6.0
          args: ["milvus", "run", "standalone"]
          env:
            - name: DEPLOY_MODE
              value: standalone
            - name: ETCD_ENDPOINTS
              value: etcd-service:2379
            - name: COMMON_STORAGE_TYPE

```

```

    value: local
  - name: MILVUS_ROOT_PASSWORD
    valueFrom:
      secretKeyRef:
        name: milvus-secret
        key: root-password
  livenessProbe:
    exec:
      command: ["curl", "-f", "http://localhost:9091/healthz"]
    initialDelaySeconds: 90
    periodSeconds: 30
    timeoutSeconds: 20
    failureThreshold: 5
  ports:
    - containerPort: 19530
      protocol: TCP
    - containerPort: 9091
      protocol: TCP
  volumeMounts:
    - name: milvus-data
      mountPath: /var/lib/milvus
  restartPolicy: Always
  volumes:
    - name: milvus-data
      persistentVolumeClaim:
        claimName: milvus-pvc
---
apiVersion: v1
kind: Service
metadata:
  name: milvus-service
spec:
  selector:
    app: milvus-standalone
  ports:
    - name: grpc
      port: 19530
      targetPort: 19530
    - name: http
      port: 9091
      targetPort: 9091

```



NOTE

- Use the gRPC port (**19530**) for the **MILVUS_ENDPOINT** setting in OGX.
- The HTTP port (**9091**) is reserved for health checks.
- If you deploy Milvus in a different namespace, use the fully qualified service name in your OGX configuration. For example: **http://milvus-service.<namespace>.svc.cluster.local:19530**

Verification

1. In the OpenShift web console, click **Workloads → Deployments**.

2. Verify that both **etcd-deployment** and **milvus-standalone** show a status of **1 of 1 pods available**.
3. Click **Pods** in the navigation panel and confirm that pods for both deployments are **Running**.
4. Click the **milvus-standalone** pod name, then select the **Logs** tab.
5. Verify that Milvus reports a healthy startup with output similar to:

```
Milvus Standalone is ready to serve ...
Listening on 0.0.0.0:19530 (gRPC)
```

6. Click **Networking** → **Services** and confirm that the **milvus-service** and **etcd-service** resources exist and are exposed on ports **19530** and **2379**, respectively.
7. (Optional) Click **Pods** → **milvus-standalone** → **Terminal** and run the following health check:

```
curl http://localhost:9091/healthz
```

A response of **{"status": "healthy"}** confirms that Milvus is running correctly.

10.3. USING POSTGRESQL IN OGX

PostgreSQL is a dependency for OGX deployments in OpenShift AI, where it serves as the mandatory metadata storage backend for supported vector storage configurations. Additionally, you can configure PostgreSQL as a remote vector database provider by enabling the pgvector extension.

Depending on your deployment requirements, these roles can be fulfilled by the same PostgreSQL instance or separate instances. For example, you might use a single instance for development and testing environments, and separate instances for production deployments that require independent scaling or isolation.

IMPORTANT

The procedures provide basic configuration suitable for development and testing. Production deployments require additional planning, including the following considerations:

- High availability and replication
- Backup and disaster recovery
- Security hardening and encryption
- Performance tuning and monitoring

10.3.1. Understanding OGX metadata storage

In OpenShift AI, OGX requires PostgreSQL as a metadata storage backend to persist state and configuration data across multiple components. Metadata storage provides durable persistence for vector stores, file management, agent state, conversation history, and other OGX services.

PostgreSQL is required as a metadata storage backend for all OpenShift AI deployments.

10.3.1.1. Role of metadata storage in OGX

OGX components require persistent storage beyond in-memory data structures. Without metadata storage, component state would be lost on pod restarts or application failures.

OGX uses metadata storage to persist:

- Vector store metadata, such as collection identifiers and document mappings.
- File metadata, including file locations, identifiers, and attributes.
- Agent state and conversation history.
- Dataset configurations and batch processing state.
- Model registry information and prompt templates.

This persistent storage allows OGX to maintain operational state across pod restarts, rescheduling, and application updates.

10.3.1.2. PostgreSQL metadata storage backends

OGX uses PostgreSQL to store multiple categories of metadata, including vector store metadata, file records, agent state, conversation history, and configuration data. These data types have different storage characteristics but are managed automatically within a single PostgreSQL instance.



IMPORTANT

PostgreSQL version 14 or later is required for all OGX deployments, including development, testing, and production environments.

If validation errors occur, confirm that the deployed OGX image version matches the configuration schema referenced by your **run.yaml**.

OGX does not provision or manage the PostgreSQL instance used for metadata storage. You must deploy and manage the PostgreSQL database and supply its connection details when deploying OGX.

10.3.2. Deploying a PostgreSQL instance with pgvector

You can connect OGX in OpenShift AI to an existing PostgreSQL instance that has the pgvector extension enabled. For development or evaluation, you can also deploy a PostgreSQL instance with the pgvector extension directly in your OpenShift project by creating Kubernetes resources through the OpenShift web console. This procedure focuses on deploying PostgreSQL with the pgvector extension for use as a remote vector store. It does not cover preparing a PostgreSQL database for use as OGX metadata storage.

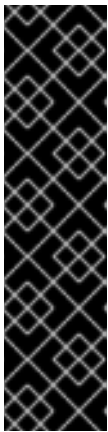
Prerequisites

- You have installed OpenShift 4.19 or newer.
- You have permissions to create resources in a project in your OpenShift cluster.
- You have PostgreSQL connection details available, including the database name, user name, and password.

- If you plan to deploy PostgreSQL in-cluster, you have a StorageClass that can provision persistent volumes.
- If you are using an existing PostgreSQL instance, the pgvector extension is installed and enabled on the target database.

Procedure

1. Log in to the OpenShift web console.
2. Select the project where you want to deploy the PostgreSQL instance.
3. Click the **Quick Create** () icon, and then click **Import YAML**.
4. Verify that the correct project is selected.
5. Copy the following YAML, replace the placeholder values, paste it into the YAML editor, and then click **Create**.



IMPORTANT

This example deploys a standalone PostgreSQL service with the pgvector extension enabled.

OGX does not automatically use this database. To use this PostgreSQL instance as a vector store, you must explicitly configure the pgvector provider in a **OGXServer**.

This example is intended for development or evaluation purposes. For production deployments, review and adapt the configuration to meet your organization's security, availability, backup, and lifecycle requirements.

Example PostgreSQL deployment with pgvector (development or evaluation)

```
apiVersion: v1
kind: Secret
metadata:
  name: <pgvector-postgresql-credentials-secret>
type: Opaque
stringData:
  POSTGRES_DB: "<database-name>"
  POSTGRES_USER: "<database-username>"
  POSTGRES_PASSWORD: "<database-password>"

---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: <pgvector-postgresql-pvc>
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
```

```

    storage: <storage-size>

---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: <pgvector-postgresql-deployment>
spec:
  replicas: 1
  selector:
    matchLabels:
      app: <pgvector-postgresql-app-label>
  template:
    metadata:
      labels:
        app: <pgvector-postgresql-app-label>
    spec:
      containers:
        - name: postgres
          image: pgvector/pgvector:pg16
          ports:
            - name: postgres
              containerPort: 5432
          env:
            - name: POSTGRES_DB
              valueFrom:
                secretKeyRef:
                  name: <pgvector-postgresql-credentials-secret>
                  key: POSTGRES_DB
            - name: POSTGRES_USER
              valueFrom:
                secretKeyRef:
                  name: <pgvector-postgresql-credentials-secret>
                  key: POSTGRES_USER
            - name: POSTGRES_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: <pgvector-postgresql-credentials-secret>
                  key: POSTGRES_PASSWORD
          volumeMounts:
            - name: pgdata
              mountPath: /var/lib/postgresql/data

      # Replace TCP socket probes with exec probes that validate SQL readiness.
      readinessProbe:
        exec:
          command:
            - /bin/sh
            - -c
            - pg_isready -h 127.0.0.1 -U "$POSTGRES_USER" -d "$POSTGRES_DB"
        initialDelaySeconds: 10
        periodSeconds: 10
        timeoutSeconds: 5
        failureThreshold: 6
      livenessProbe:
        exec:

```

```

    command:
      - /bin/sh
      - -c
      - pg_isready -h 127.0.0.1 -U "$POSTGRES_USER" -d "$POSTGRES_DB"
    initialDelaySeconds: 30
    periodSeconds: 20
    timeoutSeconds: 5
    failureThreshold: 6

    # Create the pgvector extension after PostgreSQL is actually accepting SQL.
    lifecycle:
      postStart:
        exec:
          command:
            - /bin/sh
            - -c
            - |
              set -e
              echo "Waiting for PostgreSQL to be ready before enabling pgvector..."
              until PGPASSWORD="$POSTGRES_PASSWORD" psql -h 127.0.0.1 -U
"$POSTGRES_USER" -d "$POSTGRES_DB" -c "SELECT 1" >/dev/null 2>&1; do
                sleep 2
              done
              PGPASSWORD="$POSTGRES_PASSWORD" psql -h 127.0.0.1 -U
"$POSTGRES_USER" -d "$POSTGRES_DB" -c "CREATE EXTENSION IF NOT EXISTS
vector;"

    volumes:
      - name: pgdata
        persistentVolumeClaim:
          claimName: <pgvector-postgresql-pvc>

---
apiVersion: v1
kind: Service
metadata:
  name: <pgvector-postgresql-service>
spec:
  selector:
    app: <pgvector-postgresql-app-label>
  ports:
    - name: postgres
      port: 5432
      targetPort: 5432
  type: ClusterIP

```

6. Click **Create**.

Verification

1. Navigate to **Networking → Services**.
2. Confirm that the PostgreSQL Service is listed and exposes port **5432**.
3. Navigate to **Workloads → Pods**.

4. Confirm that the PostgreSQL pod is running.



NOTE

This procedure verifies only that PostgreSQL with pgvector is deployed and reachable within the project. It does not verify integration with OGX.

10.3.3. Configuring the pgvector remote provider in OGX

To use PostgreSQL with the pgvector extension as a remote vector store, configure pgvector in your existing **OGXServer** and provide PostgreSQL connection details as environment variables. Ensure that your **OGXServer** already includes the PostgreSQL metadata storage configuration. This setup enables retrieval augmented generation (RAG) workflows in OpenShift AI by using PostgreSQL-based vector storage.

Prerequisites

- You have installed and enabled the OGX Operator in OpenShift AI.
- You have a PostgreSQL database with the pgvector extension enabled. OGX uses PostgreSQL for two purposes: metadata storage and the optional pgvector remote vector store. You can use a single PostgreSQL instance for both roles or deploy separate instances.
- You have the PostgreSQL connection details, including the host name, port number, database name, user name, and password.
- You have permissions to create Secrets and edit custom resources in your project.

Procedure

1. In the OpenShift web console, switch to the **Administrator** perspective.
2. Create a Secret that stores the PostgreSQL connection details.
 - a. Ensure that the correct project is selected.
 - b. Click **Workloads → Secrets**.
 - c. Click **Create → From YAML**.
 - d. Paste the following YAML, update the placeholder values, and then click **Create**.

Example Secret for pgvector connection details

```
apiVersion: v1
kind: Secret
metadata:
  name: pgvector-connection
type: Opaque
stringData:
  PGVECTOR_HOST: "<pgvector-hostname>"
  PGVECTOR_PORT: "<pgvector-port>"
  PGVECTOR_DB: "<database-name>"
  PGVECTOR_USER: "<database-username>"
  PGVECTOR_PASSWORD: "<database-password>"
```



IMPORTANT

The pgvector provider is not enabled automatically.

You must explicitly enable pgvector and supply its connection details through environment variables in your **OGXServer**.

In OpenShift AI, the pgvector provider is enabled when the **ENABLE_PGVECTOR** environment variable is set.

3. Update your **OGXServer** custom resource to enable pgvector and reference the Secret.
 - a. Select the **OGX Operator**.
 - b. Click the **OGXServer** tab.
 - c. Select your **OGXServer** resource.
 - d. Click **YAML**.
 - e. Update the resource to include the following fields, and then click **Save**.
Before you enable pgvector, deploy a OGX server and configure the PostgreSQL metadata store.

For more information, see [Deploying a OGX server](#).

Then update your existing **OGXServer** to add the pgvector configuration shown in the following example. The example shows only the additional environment variables required to enable the pgvector provider.

Example OGXServer configuration for pgvector

```
apiVersion: ogx.io/v1beta1
kind: OGXServer
metadata:
  name: ogx
spec:
  distribution:
    name: rh-dev
  workload:
    overrides:
      env:
        - name: ENABLE_PGVECTOR
          value: "true"
        - name: PGVECTOR_HOST
          valueFrom:
            secretKeyRef:
              name: pgvector-connection
              key: PGVECTOR_HOST
        - name: PGVECTOR_PORT
          valueFrom:
            secretKeyRef:
              name: pgvector-connection
              key: PGVECTOR_PORT
        - name: PGVECTOR_DB
          valueFrom:
```

```

    secretKeyRef:
      name: pgvector-connection
      key: PGVECTOR_DB
- name: PGVECTOR_USER
  valueFrom:
    secretKeyRef:
      name: pgvector-connection
      key: PGVECTOR_USER
- name: PGVECTOR_PASSWORD
  valueFrom:
    secretKeyRef:
      name: pgvector-connection
      key: PGVECTOR_PASSWORD

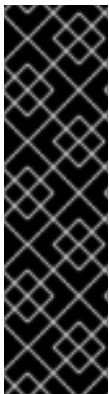
```

Verification

1. Click **Workloads → Pods**.
2. Confirm that the OGX pod restarts and reaches the **Running** state.
3. Open the pod logs and confirm that the server starts successfully and initializes the pgvector provider without errors.

10.3.4. Resources created when you create a playground

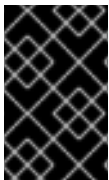
When you create a Gen AI Studio playground, OpenShift AI creates a set of Kubernetes resources to run the pgvector-enabled PostgreSQL instance for RAG. You can use the resource names and labels described here for troubleshooting or capacity planning.



IMPORTANT

The default auto-provisioned pgvector vector store for Gen AI Studio playground RAG is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).



IMPORTANT

The auto-provisioned pgvector instance is intended for development and experimentation only. For production workloads, provide your own PostgreSQL instance. For more information, see [Configure a custom PostgreSQL instance for playground RAG](#).

OpenShift AI manages the lifecycle of the following resources. All auto-provisioned resources use the naming prefix **genai-pgvector-** and carry the managed label **gen-ai.opendatahub.io/pgvector**. These resources are managed directly by the Gen AI Studio service, not by a Kubernetes operator. If a resource is manually deleted or modified, you must delete and recreate the playground to restore it.

**NOTE**

Confirm the exact resource names with your environment before scripting against them.

Table 10.1. Auto-provisioned pgvector resources

Resource type	Name	Description
Secret	genai-pgvector-credentials	Stores the auto-generated PostgreSQL connection credentials, including the database name, user, password, and host. No user input is required.
ConfigMap	genai-pgvector-init	Contains initialization scripts for the PostgreSQL instance, such as enabling the pgvector extension.
PersistentVolumeClaim	genai-pgvector-storage	Provides 5 GiB of persistent storage for vector data. Document embeddings survive pod restarts.
Deployment	genai-pgvector	Runs the PostgreSQL 16 pod with the pgvector extension using the registry.redhat.io/rhel9/postgresql-16 container image.
Service	genai-pgvector	Exposes the PostgreSQL pod on port 5432 within the namespace.
NetworkPolicy	genai-pgvector	Restricts network access to the PostgreSQL pod to traffic originating within the same namespace.

The auto-provisioned pgvector resources follow this lifecycle:

Creation

OpenShift AI creates all resources when you create a playground. Credential values are auto-generated.

Preservation on model change

When you reconfigure a playground and change the model, the pgvector instance and all uploaded document embeddings are preserved. You do not need to re-upload documents after changing models.

Deletion on playground delete

When you delete a playground, all auto-provisioned pgvector resources are removed, including the Deployment, PVC, Service, NetworkPolicy, ConfigMap, and credentials Secret. Externally configured PostgreSQL instances that you provided are not affected.

Additional resources

- [Choosing a knowledge source for playground RAG](#)
- [Configure a custom PostgreSQL instance for playground RAG](#)

10.3.5. Configure a custom PostgreSQL instance for playground RAG

You can configure the connection between the Open Gen AI Experience Toolkit (OGX) server and a pgvector-enabled PostgreSQL instance by using environment variables. For custom PostgreSQL deployments, you create a Kubernetes Secret containing the connection credentials and reference it from the OGX server configuration.

The following table describes the environment variables that configure the **remote::pgvector vector_io** provider.

Table 10.2. pgvector connection environment variables

Environment variable	Default value	Required	Description
PGVECTOR_HOST	None	Yes	Specifies the hostname or service name of the pgvector-enabled PostgreSQL instance. When this variable is set, OpenShift AI configures remote::pgvector as the default vector_io provider.
PGVECTOR_PORT	5432	No	Specifies the PostgreSQL port number.
PGVECTOR_DB	vectordb	No	Specifies the name of the PostgreSQL database for vector storage.
PGVECTOR_USER	vectoruser	No	Specifies the PostgreSQL user for vector storage connections.
PGVECTOR_PASSWORD_SECRET_NAME	None	Conditional	Specifies the name of the Kubernetes Secret that contains the pgvector password. Required for PostgreSQL instances that use password authentication. OpenShift AI injects the password from this Secret into the OGX pod environment as PGVECTOR_PASSWORD at runtime.
PGVECTOR_PASSWORD_SECRET_KEY	password	No	Specifies the key within the password Secret that contains the password value.



NOTE

The **PGVECTOR_PASSWORD** variable is not set directly. OpenShift AI reads **PGVECTOR_PASSWORD_SECRET_NAME** and **PGVECTOR_PASSWORD_SECRET_KEY** to locate a Kubernetes Secret, then injects the password value into the OGX pod environment as **PGVECTOR_PASSWORD**. The OGX server resolves **PGVECTOR_PASSWORD** at runtime. For custom deployments, create the Secret and set **PGVECTOR_PASSWORD_SECRET_NAME** to reference it.

For playground RAG, OpenShift AI automatically generates the pgvector connection credentials and stores them in the **genai-pgvector-credentials** Secret. You do not need to set these environment variables manually.

For custom PostgreSQL deployments, you create a Secret with the required connection credentials and configure the environment variables to reference it. You must set **PGVECTOR_HOST** at a minimum to enable the **remote::pgvector** provider. For PostgreSQL instances that require password authentication, you must also set **PGVECTOR_PASSWORD_SECRET_NAME**.

The following example shows a Secret for a custom PostgreSQL deployment:

```
apiVersion: v1
kind: Secret
metadata:
  name: my-pgvector-credentials
type: Opaque
stringData:
  password: <your_postgresql_password>
```

where:

<your_postgresql_password>

Specifies the password for the PostgreSQL user account that the OGX server uses to connect to your pgvector-enabled database.

After creating the Secret, you must also set at least **PGVECTOR_HOST** so that OpenShift AI can locate your PostgreSQL instance. For PostgreSQL instances that require password authentication, also set **PGVECTOR_PASSWORD_SECRET_NAME** so that OpenShift AI can inject the credentials into the OGX pod. For the complete procedure, see *Deploy a PostgreSQL instance with pgvector* in the *Configuring the Open Gen AI Experience Toolkit* guide. // TODO: Convert to xref when the target module is available.

For information about the auto-provisioned resources and their naming conventions, see [Resources created when you create a playground](#).

Additional resources

- [Choosing a knowledge source for playground RAG](#)

10.4. USING QDRANT IN OGX

Qdrant is a supported remote vector store provider for OGX in OpenShift AI. You can deploy Qdrant in your OpenShift project or connect to an existing Qdrant instance, and configure OGX to use Qdrant for retrieval-augmented generation (RAG) workloads.

To use Qdrant with OGX, complete the following tasks:

- Review how Qdrant integrates with OGX.
- Deploy a Qdrant instance or connect to an existing deployment.
- Configure your **OGXServer** to use Qdrant as the vector store provider.
- Perform vector operations through the OpenAI-compatible Vector Stores API.

10.4.1. Overview of Qdrant vector databases

Qdrant is an open source vector database optimized for high-performance similarity search and advanced filtering. In OpenShift AI, Qdrant is supported as a remote vector store provider for OGX and can be used in retrieval-augmented generation (RAG) workloads that require efficient vector indexing and durable storage.

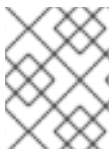
When used with OGX in OpenShift AI, Qdrant provides:

- High-performance similarity search using Hierarchical Navigable Small World (HNSW) indexing
- Filtering based on stored metadata during vector search
- Persistent storage of vector data
- Integration through the OpenAI-compatible Vector Stores API

In a RAG workflow:

- Embeddings are generated by the configured embedding provider.
- Qdrant stores embedding vectors and performs similarity search.
- OGX manages ingestion, retrieval, and model inference through a unified API.

In OpenShift AI, you must deploy Qdrant as a remote service, either within your OpenShift project or as an externally managed deployment.



NOTE

Inline Qdrant is not supported. To use Qdrant with OGX in OpenShift AI, deploy Qdrant as a remote service.

A typical remote deployment includes:

- A Qdrant service exposing HTTP (port 6333) and gRPC (port 6334) endpoints
- Persistent storage for vector data
- Optional API key authentication

For deployment and configuration instructions, see [Using Qdrant in OGX](#).

10.4.2. Deploying a Qdrant vector database

You can connect OGX in OpenShift AI to an existing Qdrant instance or deploy a Qdrant vector database in your OpenShift project. For development or evaluation purposes, you can deploy Qdrant by creating Kubernetes resources in the OpenShift web console.

Prerequisites

- You have installed OpenShift 4.19 or later.
- You have permission to create resources in a project.

- A StorageClass is available that can provision a PersistentVolume for the PersistentVolumeClaim used by this deployment.



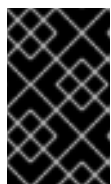
NOTE

This example uses a single PersistentVolumeClaim. If your cluster uses dynamic provisioning, the StorageClass provisions the required PersistentVolume automatically.

- Optional: You have an API key for Qdrant authentication. If your Qdrant instance does not require authentication, remove the Secret and the **QDRANT__SERVICE__API_KEY** environment variable from the deployment example.

Procedure

1. Log in to the OpenShift web console.
2. From the **Project** list, select the project where you want to deploy Qdrant.
3. Click **Import YAML**.
4. Paste the following YAML:



IMPORTANT

This example deploys a standalone Qdrant service for development or evaluation. For production deployments, review and adapt the configuration to meet your organization's security, availability, backup, and lifecycle requirements.

```
apiVersion: v1
kind: Secret
metadata:
  name: <qdrant_credentials_secret>
type: Opaque
stringData:
  QDRANT_API_KEY: "<api_key>"

---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: <qdrant_pvc>
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: <storage_size>

---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: <qdrant_deployment>
spec:
```

```

replicas: 1
selector:
  matchLabels:
    app: <qdrant_app_label>
template:
  metadata:
    labels:
      app: <qdrant_app_label>
  spec:
    containers:
      - name: qdrant
        image: qdrant/qdrant:v1.12.0
        ports:
          - name: http
            containerPort: 6333
          - name: grpc
            containerPort: 6334
        env:
          - name: QDRANT__SERVICE__API_KEY
            valueFrom:
              secretKeyRef:
                name: <qdrant_credentials_secret>
                key: QDRANT_API_KEY
        volumeMounts:
          - name: qdrant-storage
            mountPath: /qdrant/storage
          - name: qdrant-storage
            mountPath: /qdrant/snapshots
            subPath: snapshots
        readinessProbe:
          httpGet:
            path: /readyz
            port: 6333
            initialDelaySeconds: 5
            periodSeconds: 10
        livenessProbe:
          httpGet:
            path: /healthz
            port: 6333
            initialDelaySeconds: 10
            periodSeconds: 20
        volumes:
          - name: qdrant-storage
            persistentVolumeClaim:
              claimName: <qdrant_pvc>

---
apiVersion: v1
kind: Service
metadata:
  name: <qdrant_service>
spec:
  selector:
    app: <qdrant_app_label>
  ports:
    - name: http

```

```

port: 6333
targetPort: 6333
- name: grpc
  port: 6334
  targetPort: 6334
  type: ClusterIP

```



NOTE

If your Qdrant instance does not require authentication, remove the Secret and the **QDRANT__SERVICE__API_KEY** environment variable from the Deployment configuration.

5. Replace the placeholder values as follows:

- **<qdrant_credentials_secret>**: A name for the Secret that stores the Qdrant API key, for example **qdrant-credentials**.
- **<api_key>**: An API key for authenticating with Qdrant. If authentication is not required, remove the Secret and the **QDRANT__SERVICE__API_KEY** environment variable from the Deployment.
- **<qdrant_pvc>**: A name for the PersistentVolumeClaim, for example **qdrant-pvc**.
- **<storage_size>**: The storage capacity to request, for example **10Gi**.
- **<qdrant_deployment>**: A name for the Deployment, for example **qdrant**.
- **<qdrant_app_label>**: A label for the application, for example **qdrant**.
- **<qdrant_service>**: A name for the Service, for example **qdrant-service**.

6. Click **Create**.

Verification

- The Qdrant Service is present in the project and exposes ports **6333** (HTTP) and **6334** (gRPC). You can confirm this on the **Networking → Services** page in the OpenShift web console.
- The Qdrant pod reaches the **Running** state. You can confirm this on the **Workloads → Pods** page in the OpenShift web console.



NOTE

This verification confirms only that Qdrant is deployed and reachable within the project. To use this Qdrant instance with OGX, configure the Qdrant provider in a **OGXServer**.

10.4.3. Configuring the Qdrant remote provider in OGX

To use Qdrant as a remote vector store, configure your **OGXServer** resource with the connection details for your Qdrant service. This configuration enables OGX to store and retrieve embedding vectors using Qdrant in OpenShift AI.

Prerequisites

- You have installed and enabled the OGX Operator in OpenShift AI.
- You have a running Qdrant instance that is accessible from your OpenShift cluster.
- You have the Qdrant connection details, including the service URL and, if required, an API key.
- You have permission to create Secrets and modify custom resources in your project.

Procedure

1. In the OpenShift web console, switch to the **Administrator** perspective.
2. Create a Secret that stores the Qdrant connection details used by OGX. This Secret must contain the URL of the Qdrant service and, if required, the API key.



NOTE

If you deployed Qdrant by using the procedure in [Deploying a Qdrant vector database](#), create this Secret separately for the OGX configuration. The Secret created during the Qdrant deployment does not contain the **QDRANT_URL** value required by the OGX provider.

- a. From the **Project** list, select the project where the **OGXServer** resource is deployed.
- b. Click **Workloads → Secrets**.
- c. Click **Create → From YAML**.
- d. Paste the following YAML:

```
apiVersion: v1
kind: Secret
metadata:
  name: qdrant-connection
type: Opaque
stringData:
  QDRANT_URL: "<qdrant_url>"
  QDRANT_API_KEY: "<api_key>"
```

- e. Replace the placeholder values as follows:
 - **<qdrant_url>**: The full URL to the Qdrant service, for example **http://qdrant-service:6333**. For in-cluster deployments, use the Service name and port. For external deployments, use the external URL.
 - **<api_key>**: The API key for authenticating with Qdrant. If authentication is not enabled for your Qdrant instance, remove the **QDRANT_API_KEY** entry from both the Secret and the **env** section in the **OGXServer** configuration.
 - f. Click **Create**.
3. Update your **OGXServer** custom resource to reference the Secret and supply the required environment variables.
 4. Go to the **Installed Operators** page. The navigation path depends on your OpenShift version:

- On OpenShift 4.20 and later, click **Ecosystem** → **Installed Operators**.
 - On OpenShift 4.19, click **Operators** → **Installed Operators**.
- a. Select the **OGX Operator**.
 - b. Click the **OGXServer** tab.
 - c. Select your **OGXServer** resource.
 - d. Click **YAML**.
 - e. Update the resource to include the following fields.

**NOTE**

The environment variable names and configuration fields used by the Qdrant provider can vary depending on the OGX version included with OpenShift AI. Before applying this configuration, verify that the variables and fields match the supported versions listed in [Supported Configurations for 3.x](#).

```
apiVersion: ogx.io/v1beta1
kind: OGXServer
metadata:
  name: ogx
spec:
  server:
    containerSpec:
      env:
        - name: ENABLE_QDRANT
          value: "true"
        - name: QDRANT_URL
          valueFrom:
            secretKeyRef:
              name: qdrant-connection
              key: QDRANT_URL
        - name: QDRANT_API_KEY
          valueFrom:
            secretKeyRef:
              name: qdrant-connection
              key: QDRANT_API_KEY
```

- f. Click **Save**.

Verification

- The OGX pod reaches the **Running** state. You can confirm this on the **Workloads** → **Pods** page in the OpenShift web console.
 - The pod logs show that the Qdrant provider initializes successfully and does not report connection errors.
 - Vector operations executed through the OGX API complete successfully, confirming that OGX can communicate with Qdrant.
- For information about performing vector operations, see:

[Performing vector operations with Qdrant](#) .

10.4.4. Performing vector operations with Qdrant

After configuring Qdrant as the vector store provider in OGX, you can perform vector operations by using the OpenAI-compatible Vector Stores API exposed by OGX. These operations include creating vector stores, adding documents, performing similarity search, and deleting vector stores. You interact with the OGX API rather than connecting directly to Qdrant. OGX manages collection creation, embedding generation, and query execution on your behalf.

Prerequisites

- You have installed and enabled the OGX Operator in OpenShift AI.
- You have configured Qdrant as the vector store provider in your **OGXServer**.
- You have an embedding model available through a configured inference provider.
- You have network access to the OGX API endpoint.
- You have installed the **jq** command-line utility.
For installation instructions, see [jq](#).
- You have the **curl** command-line tool installed.

Procedure

1. Determine how you will access the OGX API.
You can access the API from within the cluster or from outside the cluster.
 - **In-cluster access:** Run the **curl** commands from a pod in the same project, or from a workstation that has network access to the OGX Service.
 - **External access:** Expose the OGX Service by creating a Route, and then use the Route URL from your local workstation.
For this procedure, set **OGX_URL** to the service or route root URL **without** the **/v1** suffix. The example commands append **/v1** as part of the endpoint path.

For more information about API compatibility and base URL requirements, see

[OpenAI compatibility for RAG APIs in OGX](#) .

Example base URL for in-cluster access

```
OGX_URL="http://ogx-service:8321"
```

Example base URL for external access through a Route

```
OGX_URL="https://ogx-route.example.com"
```

2. Create a vector store and capture its ID.

```
CREATE_RESPONSE=$(curl -s -X POST "${OGX_URL}/v1/vector_stores" \
-H "Content-Type: application/json" \
```

```
-d '{
  "name": "my-rag-store",
  "embedding_model": "vllm/ibm-granite/granite-embedding-125m-english",
  "embedding_dimension": 768,
  "provider_id": "qdrant-remote"
}'

VECTOR_STORE_ID=$(echo "$CREATE_RESPONSE" | jq -r '.id')
echo "Vector store ID: ${VECTOR_STORE_ID}"
```

Ensure that the **VECTOR_STORE_ID** variable contains a valid value before continuing.

10.4.4.1. Add files to a vector store

Upload files to the vector store for ingestion. OGX automatically splits the content into chunks, generates embeddings, and stores them in Qdrant.

Example using curl

```
FILE_RESPONSE=$(curl -s -X POST "${OGX_URL}/v1/vector_stores/${VECTOR_STORE_ID}/files" \
  -F "file=@/path/to/document.pdf" \
  -F "purpose=assistants")

FILE_ID=$(echo "$FILE_RESPONSE" | jq -r '.id')
echo "File ID: ${FILE_ID}"
```

10.4.4.2. Query a vector store

Perform similarity search to retrieve relevant content from the vector store. The search query is converted into an embedding and compared with stored vectors in Qdrant.

Example using curl

```
curl -X POST "${OGX_URL}/v1/vector_stores/${VECTOR_STORE_ID}/search" \
  -H "Content-Type: application/json" \
  -d '{
    "query": "What is retrieval-augmented generation?",
    "max_results": 5
  }'
```

10.4.4.3. Delete a vector store

Delete a vector store when it is no longer required. This removes the vector store and its associated data from Qdrant.

Example using curl

```
curl -X DELETE "${OGX_URL}/v1/vector_stores/${VECTOR_STORE_ID}"
```

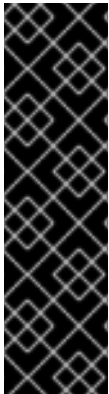
Verification

- Creating a vector store returns a valid vector store ID.

- File uploads complete successfully and are accepted by the API.
- Search queries return results from the ingested content.

10.5. CONFIGURE EXTERNAL VECTOR STORES FOR THE PLAYGROUND

As an administrator, you can make external vector stores available to users in the Gen AI Studio playground. Users can then select these vector stores as knowledge sources for RAG instead of uploading files.



IMPORTANT

External vector stores for RAG in the Gen AI Studio playground is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

Prerequisites

- You have installed and enabled the OGX Operator in OpenShift AI.
- You have one or more deployed vector databases with data already ingested. Supported databases are PostgreSQL with pgvector, Qdrant, and Milvus.
- You have the connection details for each vector database, including the hostname, port, and credentials.
- You know the embedding model and embedding dimension used to create the vector data in each store.
- You have permissions to create ConfigMaps and Secrets in the target project namespace.

Procedure

1. In the OpenShift web console, switch to the **Administrator** perspective and select the project namespace where the playground is configured.
2. Create a Secret for each vector database that requires authentication.
 - a. Click **Workloads → Secrets**.
 - b. Click **Create → From YAML**.
 - c. Paste the following YAML, update the placeholder values, and then click **Create**.

Example Secret for a pgvector-enabled PostgreSQL database

```
apiVersion: v1
kind: Secret
```

```

metadata:
  name: <my_vector_store_credentials>
  type: Opaque
  stringData:
    password: <my_database_password>

```

- **<my_vector_store_credentials>** specifies a name for the Secret. Use this name in the ConfigMap **secretRefs** field.
- **<my_database_password>** specifies the password for the vector database. Repeat this step for each vector database that requires credentials.

3. Create the **gen-ai-aa-vector-stores** ConfigMap.

- Click **Workloads** → **ConfigMaps**.
- Click **Create ConfigMap**.
- Paste the following YAML, update the placeholder values, and then click **Create**.

Example ConfigMap for external vector stores

```

apiVersion: v1
kind: ConfigMap
metadata:
  name: gen-ai-aa-vector-stores
data:
  config.yaml: |
    providers:
      vector_io:
        - provider_id: <my_provider_id>
          provider_type: <my_provider_type>
          config:
            host: <my_database_host>
            port: <my_database_port>
            db: <my_database_name>
            user: <my_database_user>
            custom_gen_ai:
              credentials:
                secretRefs:
                  - name: <my_vector_store_credentials>
                    key: password

    registered_resources:
      vector_stores:
        - provider_id: <my_provider_id>
          vector_store_id: <my_vector_store_id>
          embedding_model: <my_embedding_model>
          embedding_dimension: <my_embedding_dimension>
          vector_store_name: "<my_display_name>"
          metadata:
            description: "<my_vector_store_description>"

```

- **<my_provider_id>** specifies a unique identifier for this vector database provider, such as **pgvector-prod** or **qdrant-docs**.

- **<my_provider_type>** specifies the provider type. Supported values are **remote::pgvector**, **remote::qdrant**, and **remote::milvus**.
 - **<my_database_host>** specifies the hostname or service name of the vector database.
 - **<my_database_port>** specifies the port number for the vector database.
 - **<my_database_name>** specifies the database name. Applicable to **remote::pgvector** providers.
 - **<my_database_user>** specifies the database user. Applicable to **remote::pgvector** providers.
 - **<my_vector_store_credentials>** specifies the name of the Secret that contains the database credentials. Must match the Secret you created in the previous step.
 - **<my_vector_store_id>** specifies a unique identifier for the vector store collection, such as **vs_product-embeddings**.
 - **<my_embedding_model>** specifies the embedding model used to create the vector data, such as **ibm-granite/granite-embedding-125m-english**.
 - **<my_embedding_dimension>** specifies the dimension of the embedding vectors, such as **768**. This value must match the dimension used when the data was ingested.
 - **<my_display_name>** specifies the vector store name that users will see in Gen AI Studio. For example, in AI Asset Endpoints, playground.
 - **<my_vector_store_description>** specifies an optional description of the vector store. To add multiple vector stores, add entries to both the **providers.vector_io** list and the **registered_resources.vector_stores** list.
4. Enable the external vector stores feature in the OpenShift AI dashboard configuration. Run the following command to set the **externalVectorStores** feature flag to **true** in the **OdhDashboardConfig** custom resource:

```
$ oc patch OdhDashboardConfig odh-dashboard-config -n redhat-ods-applications --type=merge -p '{"spec":{"dashboardConfig":{"externalVectorStores":true}}}'
```

Verification

- In the OpenShift AI dashboard, click **Gen AI studio → Playground**.
- In the settings panel, click the **Knowledge** tab.
- Enable the **RAG** toggle.
- Select **Use an existing vector store**. The vector stores you configured appear in the list.

Additional resources

- [Choosing a knowledge source for playground RAG](#)
- [Use an external vector store for RAG in the playground](#)

CHAPTER 11. DEPLOY OGX FOR MULTI-TENANCY

As an OpenShift cluster administrator, you can use OGX to deploy a single server or cluster that manages multiple tenant deployments.

11.1. OVERVIEW OF MULTI-TENANCY ON OGX

Multi-tenancy allows teams to share infrastructure while isolating data and access. Without multi-tenancy controls, any authenticated user can view, modify or delete any other user's resources or applications.

11.1.1. Single-server vs Multi-server environments

Single-server: A single OGX server serves all tenants. Tenant administrators manage the configurations on the shared server and provision resources to tenant users. Isolation is enforced at the application layer with JSON Web Token (JWT) validation and Attribute-Based Access Control (ABAC).

When to use single-server multi-tenancy

Single-server multi-tenancy is recommended when:

- Teams share a cluster and trust the platform, but require data separation.
- Environments where teams can share a single database, a single set of models, and a unified pod.
- Fast deployment is necessary without the overhead of provisioning new infrastructure.

Multi-server: The OGX Operator deploys multiple **OGXServer** custom resources (CRs) within tenant admin namespaces. Isolation is enforced at the infrastructure level with Role-Based Access Control (RBAC), **NetworkPolicies**, and **ResourceQuotas**.

When to use multi-server multi-tenancy

Multi-server multi-tenancy is recommended when:

- Tenant admins need full infrastructure isolation including, separate pods, databases, and storage.
- Environments with strict compliance where process-level separation is mandated.
- Tenant admins need different OGX configurations including different providers, models, and policies.
- Tenant admins want to manage their own **OGXServer** CR and the operator reconciles into a dedicated deployment, service, and storage.

11.1.2. Roles for multi-tenancy environments

In OpenShift AI, there are various roles that manage or interact with the resources the single or multi server provides. A tenant consists of one or more namespaces that own **OGXServer** custom resources (CRs). By default, resources are isolated within the tenants designated namespace.

- **Platform Admin:** Operates at the cluster or multi-server level, the platform admin installs the operator, provisions tenant namespaces, configures Role-Based Access Control (RBAC), **ResourceQuotas** CRs, and manages CRDs.
- **Tenant Admin:** Operates at the namespace or single-server level, the tenant admin creates and manages **OGXServer** CRs, configures OGX providers, secrets, and networking configurations.
- **Tenant User:** Operates at the API level, the tenant user makes requests to an API endpoint without OpenShift cluster access.

11.1.3. Operator-enforced isolation

The OGX Operator enforces security and administrative boundaries. By default, the operator enforces the following isolation boundaries:

- **Namespace-scoped resources:** All resources created by the operator, including **Deployment**, **Service**, **ServiceAccount**, **RoleBinding**, **NetworkPolicy**, or **PersistentVolumeClaim**, are created in the CR namespace.
- **ConfigMap/ Secret references:** All **ConfigMap** and secret references in the custom resource specifications are restricted to the CR's namespace.
- **Network Isolation:** The **NetworkPolicy** specification is created for each distribution with the following defaults:

Table 11.1. Network Isolation Rules

Direction	Default Rule	Configurable
Ingress	Allow from same namespace	Yes, with the allowedFrom parameter
Ingress	Allow from operator namespace	No, required for operator health checks
Ingress	Deny all other	Yes, with the allowedFrom parameter
Egress	Unrestricted or no policy	Yes, via egress rules
Egress	Auto-include DNS (port 53)	No, always injected when egress rules are present

- **Server pod permissions:** The server **ServiceAccount** has no Kubernetes API permissions. Secrets are injected as environment variables via **secretKeyRef**, not read at runtime by the server.

11.2. CREATING A SINGLE-SERVER MULTI-TENANT ENVIRONMENT

A single-server multi-tenant environment allows you to run a single OGX server with multiple users connecting to a single namespace.

As a tenant admin, you need to configure security boundaries, mapping identities, and networking configurations.

Supported authentication providers

- **OAuth2 JWKS:** Validates JWT with a JWKS endpoint, best used for Kubernetes OIDC, Keycloak and standard OIDC providers.
- **OAuth2 Introspection:** Validates tokens via RFC 7662, best used for legacy OAuth servers.
- **Kubernetes:** Validates via the K8s **SelfSubjectReview** API, best used for native in-cluster service accounts.
- **GitHub:** Validates GitHub PATs with the GitHub API, best used for open-source or deployments in GitHub environments.
- **Upstream Header:** Reads identity from gateway headers, best used for authorino, istio or API gateway setups.
- **Custom:** Forwards the token to a user-provided HTTP endpoint, best used for specific proprietary integrations.

Authorization Layers

OGX enforces security through two distinct authorization mechanisms:

Access Policy (ABAC) - This policy controls which specific resources a tenant user can create, read, update, and delete. This policy can be modified to allow team-based sharing.

Optional: **Route Policy (RBAC)** - OGX checks if the tenant users role is allowed to use the requested URL. For example, some tenant users can be restricted to inference endpoints while admins maintain full access



IMPORTANT

Route policy capability is available in OpenShift AI but is not included in the default distribution configuration. Utilization requires deploying custom **config.yaml** configuration.

Each OGX resource has different Isolation levels.

Table 11.2. Resource Isolation Matrix

Resource Name	Isolation Level	Sharing / Access Model
Responses	Isolated (ABAC)	Fully private, each tenant sees only their own data.
Files	Isolated (ABAC)	Fully private, each tenant sees only their own data.
Vector Stores	Isolated (ABAC)	Fully private, each tenant sees only their own data.
Batches	Isolated (ABAC)	Fully private, each tenant sees only their own data.
Interactions	Isolated (ABAC)	Fully private, each tenant sees only their own data.

Resource Name	Isolation Level	Sharing / Access Model
Inference Store	Isolated (ABAC)	Fully private, each tenant sees only their own data.
Models	Shared (Configurable)	Infrastructure resource, available to all tenants by default.
Tool Groups	Shared (Configurable)	Available to all tenants by default, but can be restricted per-team via access policy.

The following procedures describe how to set up an OGX server with custom authentication and routing

Prerequisites

- You have installed the OpenShift CLI (**oc**)
- You have installed the OGX Operator on your OpenShift AI cluster.

Procedure

1. Deploy an **OGXServer** CR with your custom configuration. For more information, see "Deploying an OGX Server".
2. Configure claims mapping in the **config.yaml** file

```
server:
  auth:
    provider_config:
      claims_mapping:
        realm_access.roles: roles
      groups: teams
      tenant_id: namespaces
```

3. You now need to enable authentication on your server. You can enable Kubernetes OIDC, which is recommended for in-cluster workloads, or Keycloak for basic users and external clients.

- **Enabling authorization with Kubernetes OIDC**

- a. Get the clusters OIDC endpoints with the following command:

```
$ AUTH_ISSUER=$(oc get --raw /.well-known/openid-configuration | jq .issuer -r)
$ AUTH_JWKS_URI=$(oc get --raw /.well-known/openid-configuration | jq .jwks_uri -r)
```

- b. Set the following additional environment variables:

```
AUTH_AUDIENCE=ogx
AUTH_VERIFY_TLS=true
```

- c. The default distribution configuration activates when you set the necessary environment variables.

Default auth configuration YAML

```
auth:
  provider_config:
    type: ${env.AUTH_ISSUER:+oauth2_token} # activates only when
AUTH_ISSUER is set
    audience: ${env.AUTH_AUDIENCE:=ogx}
    issuer: ${env.AUTH_ISSUER:=}
  jwks:
    uri: ${env.AUTH_JWKS_URI:=}
    verify_tls: ${env.AUTH_VERIFY_TLS:=true}
```

- **Enabling authorization with Keycloak**

- a. Set your Keycloak details in the **OGXServer** custom resource

```
apiVersion: ogx.io/v1beta1
kind: OGXServer
metadata:
  name: ogx-shared-server
  namespace: ogx-system
spec:
  env:
    - name: AUTH_ISSUER
      value: "https://keycloak.example.com/realms/my-org"
    - name: AUTH_JWKS_URI
      value: "https://keycloak.example.com/realms/my-org/protocol/openid-
connect/certs"
    - name: AUTH_AUDIENCE
      value: "ogx-api"
```

- 4. You then need to configure the access policy of your server. OpenShift AI ships a default access policy that provides owner-based isolation.

Default access policy YAML

```
auth:
  access_policy:
    # System resources are readable by all
    - permit:
        actions: [read]
        when: resource is unowned
        description: "All users can read system resources"
    # Any authenticated user can create resources
    - permit:
        actions: [create]
        description: "Authenticated users can create resources"
    # Only the owner can read, update, or delete resources
    - permit:
        actions: [read, update, delete]
        when: user is owner
        description: "Owners can manage their own resources"
```

- a. For team-based sharing where users can see the resources of users on the same team, use the following example **access_policy** config:

```
auth:
  access_policy:
    - permit:
        actions: [read]
        when: resource is unowned
        description: "All users can read system resources"
    - permit:
        actions: [create]
        description: "Authenticated users can create resources"
    - permit:
        actions: [read, update, delete]
        when: user is owner
        description: "Owners can manage their own resources"
    - permit:
        actions: [read]
        when: user in owners teams
        description: "Team members can read each others resources"
```

- b. For permissions where an admin can access everything, use the following example **access_policy** config:

```
auth:
  access_policy:
    # Admin bypass: full access to all resources
    - permit:
        actions: [create, read, update, delete]
        when: user with admin in roles
        description: "Admins have full access"
    # Standard user policies
    - permit:
        actions: [read]
        when: resource is unowned
    - permit:
        actions: [create]
    - permit:
        actions: [read, update, delete]
        when: user is owner
```

5. Create the project namespaces, for example:

```
$ oc new-project <project-a>
$ oc new-project <project-b>
```

6. Set the service accounts for each role, for example

```
$ oc create serviceaccount ogx-developer -n <project-a>
$ oc create serviceaccount ogx-agent -n <project-b>
```

7. The tenant admin is responsible for provisioning auth tokens for the tenant users. You can generate the tokens using Kubernetes OIDC, which is recommended for cluster workloads, or Keycloak for basic users or external clients.

- Accessing a Kubernetes OIDC token

- a. You can create a token based on team roles and project names:

```
$ TOKEN=$(oc create token ogx-developer -n <project-a> --audience ogx --
duration=3600s)
```

- Accessing a Keycloak token

- a. Configure your **config.yaml** file to trust KeyCloak

```
auth.provider_config.issuer: https://keycloak.example.com/realms/ai-platform
auth.provider_config.jwks.uri: https://keycloak.example.com/realms/ai-
platform/protocol/openid-connect/certs
```

- b. Access and set the token environment variable:

```
TOKEN=$(curl -s -X POST \
  "https://keycloak.example.com/realms/ai-platform/protocol/openid-connect/token" \
  -d "grant_type=password&client_id=ogx&username=alice&password=****" \
  | jq -r .access_token)
```

8. Your tenant users can now access the resources on the namespace. For more information, see "Using APIs as a tenant user".

11.3. CREATING A MULTI-SERVER MULTI-TENANT ENVIRONMENT

In OpenShift AI, the platform administrator can set up a multi-server multi-tenant environment, enabling tenant admins to manage namespaces for their respective tenant users.

Platform admin responsibilities

- **Namespace Provisioning:** Create and label tenant namespaces before CR creation.
- **RBAC:** Create **Roles** and **RoleBindings** per tenant namespace
- **ResourceQuota/ LimitRange:** Set per-namespace compute and object quotas.
- **Monitoring:** Monitor tenant resource usage and set alerts for quota pressure.
- **(Optional) Network enforcement:** Enable network isolation requirements.

Tenant admin responsibilities

- **Distribution configuration:** Create and manage **OGXServer** CRs in their namespaces.
- **Resource limits:** Configure **resources.requests**, **resources.limits**, and **maxReplicas** on CRs. The **ResourceQuota** CR provisioned by the platform admin enforces that these configurations stay within their namespace quota.
- **Secrets:** Create and manage Secrets including, API keys and provider credentials, in their namespace.
- **Network ingress:** Configure **spec.network.policy.ingress** to allow access from specific users.

- **Network egress:** Configure **spec.network.policy.egress** to restrict outbound traffic from server pods.

The following procedure displays the necessary CR configurations that the platform admin needs to enable for the tenant admin.

Prerequisites

- You have cluster administrator permissions.
- You have installed the OpenShift CLI (**oc**)
- You have installed the OGX Operator on your OpenShift AI cluster.

Procedure

1. Create a **ClusterRole** custom resource for configuring the platform admin permissions.

Example platform admin ClusterRole CR

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: ogx-platform-admin
rules:
- apiGroups: ["ogx.io"]
  resources: ["ogxservers"]
  verbs: ["get", "list", "watch", "create", "update", "patch", "delete"]
- apiGroups: [""]
  resources: ["namespaces"]
  verbs: ["get", "list", "watch", "create"]
- apiGroups: ["rbac.authorization.k8s.io"]
  resources: ["roles", "rolebindings"]
  verbs: ["get", "list", "watch", "create", "update", "patch"]
```

2. Grant the tenant admin permissions to manage CR in their namespace with the following example **Role** configurations:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: ogx-tenant-admin
  namespace: tenant-a
rules:
- apiGroups: ["ogx.io"]
  resources: ["ogxservers"]
  verbs: ["get", "list", "watch", "create", "update", "patch", "delete"]
- apiGroups: [""]
  resources: ["secrets", "configmaps"]
  verbs: ["get", "list", "watch", "create", "update", "patch", "delete"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: ogx-tenant-admin
```

```

namespace: tenant-a
subjects:
- kind: Group
  name: tenant-a-admins
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: ogx-tenant-admin
  apiGroup: rbac.authorization.k8s.io

```

3. Enable **ResourceQuota** CR using the following configurations as an example:

```

apiVersion: v1
kind: ResourceQuota
metadata:
  name: ogx-tenant-quota
  namespace: tenant-a
spec:
  hard:
    requests.cpu: "8"
    requests.memory: 32Gi
    limits.cpu: "16"
    limits.memory: 64Gi
    pods: "10"

```

4. Enable preferred **NetworkPolicy** CR configurations:

- Allow specific namespace

```

spec:
  network:
    policy:
      ingress:
      - from:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: frontend-ns
      ports:
      - port: 8321
        protocol: TCP

```

- Allow namespace by label

```

spec:
  network:
    policy:
      ingress:
      - from:
        - namespaceSelector:
            matchLabels:
              tenant: customer-a
      ports:
      - port: 8321
        protocol: TCP

```

- Allow specific pods in a namespace

```
spec:
  network:
    policy:
      ingress:
        - from:
            - namespaceSelector:
                matchLabels:
                  kubernetes.io/metadata.name: frontend-ns
            podSelector:
                matchLabels:
                  app: api-gateway
        ports:
          - port: 8321
            protocol: TCP
```

- Allow external CIDR

```
spec:
  network:
    policy:
      ingress:
        - from:
            - ipBlock:
                cidr: 10.0.0.0/8
                except:
                  - 10.0.1.0/24
        ports:
          - port: 8321
            protocol: TCP
```

Verification

- Log in to the cluster as a tenant administrator and verify that you can manage resources in your designated namespace.

11.4. USING APIS AS A TENANT USER

You can access and use various APIs configured by a platform or tenant admin.

Procedure

1. Obtain your credentials from the tenant admin. Your tenant admin will provide you with:

- An OGX endpoint URL, for example:

```
https://ogx.apps.cluster.example.com/v1
```

- An API Key or token. You can obtain the keys in various ways:

- a. An environment variable: The **OGX_API_KEY** is already set by the platform admin when you open the notebook.

- b. Keycloak login: You login to keycloak and the OGX SDK is responsible for refreshing.
 - c. API key: Tenant admin generates and provides you with the key that you use in your configuration.
2. You can now access the APIs in the namespaces

Example using the APIs

```
OGX_URL="https://ogx.apps.cluster.example.com"
TOKEN("<your-api-key-from-admin>")

# List models
curl -s -H "Authorization: Bearer $TOKEN" "$OGX_URL/v1/models" | python3 -m json.tool

# Create a response
curl -s -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  "$OGX_URL/v1/responses" \
  -d '{"model": "vllm-inference/llama-3-2-3b", "input": "Summarize quantum
computing", "store": true}' \
  | python3 -m json.tool

# List your responses
curl -s -H "Authorization: Bearer $TOKEN" "$OGX_URL/v1/responses" | python3 -m json.tool

# Upload a file (owned by you)
curl -s -H "Authorization: Bearer $TOKEN" \
  -F "file=@dataset.jsonl" -F "purpose=assistants" \
  "$OGX_URL/v1/files" | python3 -m json.tool

# Create a vector store (owned by you)
curl -s -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  "$OGX_URL/v1/vector_stores" \
  -d '{"name": "my-knowledge-base"}' | python3 -m json.tool
```

Example using the Python SDK

```
import os
from openai import OpenAI

# Token and URL are provided by your platform admin
# (typically pre-set as environment variables in your notebook/workspace)
client = OpenAI(
    base_url=os.environ.get("OGX_URL", "https://ogx.apps.cluster.example.com/v1"),
    api_key=os.environ["OGX_API_KEY"],
)

# Create a stored response — automatically owned by your identity
response = client.responses.create(
    model="vllm-inference/llama-3-2-3b",
    input="Explain transformers",
    store=True,
)
```

```
# List responses — only returns yours, other teams' responses are invisible  
my_responses = client.responses.list()
```

CHAPTER 12. CONFIGURING OGX WITH OAUTH AUTHENTICATION

You can configure OGX to use role-based access control (RBAC) for model access with OAuth authentication on OpenShift AI. The following example shows how to configure OGX so that all authenticated users can access a vLLM model, while only specific users can access an OpenAI model. This example uses Keycloak to issue and validate tokens.

This procedure assumes that the Keycloak server is available at <https://my-keycloak-server.com>.



IMPORTANT

When you access OGX APIs, the required base URL depends on the client that you use.

- For OpenAI-compatible clients or raw HTTP requests, include the **/v1** path suffix in the base URL.
For example, **<http://ogx-service:8321/v1>**
- For the **OGXClient** SDK, do not include the **/v1** path suffix in the base URL.
For example, **<http://ogx-service:8321>**

If you use an incorrect base URL, requests fail.

Prerequisites

- You have installed OpenShift 4.19 or later.
- You have logged in to Red Hat OpenShift AI.
- You have cluster-admin privileges for your OpenShift cluster.
- You have a Keycloak instance configured with the following settings:
 - Realm: **ogx-demo**
 - Client: **ogx** with direct access grants enabled
 - Role: **inference_max**
 - A protocol mapper that adds realm roles to the access token under the **ogx_roles** claim
 - Two test users:
 - **user1** with no assigned roles
 - **user2** assigned the **inference_max** role
- You have saved the Keycloak client secret for token requests.
- Your Keycloak server is reachable at <https://my-keycloak-server.com>.
- You have installed the OpenShift CLI (**oc**) as described in the documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform

- [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS

Procedure

1. To configure OGX to use role-based access control (RBAC) for model access, view and verify the OAuth provider token structure.

- a. Generate a Keycloak test token by running the following command:

```
$ curl -d client_id=ogx -d client_secret=YOUR_CLIENT_SECRET -d username=user1 -d password=user-password -d grant_type=password https://my-keycloak-server.com/realms/ogx-demo/protocol/openid-connect/token | jq -r .access_token > test.token
```

- b. View the token claims by running the following command:

```
$ cat test.token | cut -d . -f 2 | base64 -d 2>/dev/null | jq .
```

Example token structure from Keycloak

```
{
  "iss": "https://my-keycloak-server.com/realms/ogx-demo",
  "aud": "account",
  "sub": "761cdc99-80e5-4506-9b9e-26a67a8566f7",
  "preferred_username": "user1",
  "ogx_roles": [
    "inference_max"
  ]
}
```

2. Update your existing **run.yaml** file to add the OAuth parameters.

Example OAuth parameters in the run.yaml file

```
server:
  port: 8321
  auth:
    provider_config:
      type: "oauth2_token"
    jwks:
      uri: "https://my-keycloak-server.com/realms/ogx-demo/protocol/openid-connect/certs"
      1
    key_recheck_period: 3600
    issuer: "https://my-keycloak-server.com/realms/ogx-demo" 2
    audience: "account"
    verify_tls: true
    claims_mapping:
      ogx_roles: "roles" 3
    access_policy:
      - permit: 4
        actions: [read]
        resource: model::vllm-inference/llama-3-2-3b
        description: Allow all authenticated users to access the Llama 3.2 model
```

```
- permit: 5
  actions: [read]
  resource: model::openai/gpt-4o-mini
  when: user with inference_max in roles
  description: Allow only users with the inference_max role to access OpenAI models
```

1 2 Specify your Keycloak host and realm in the URL.

3 Maps the **ogx_roles** claim from the token to the **roles** field.

4 Allows all authenticated users to access vLLM models.

5 Restricts OpenAI models to users with the **inference_max** role.

3. Create a ConfigMap that uses the updated **run.yaml** configuration by running the following command:

```
$ oc create configmap ogx-custom-config --from-file=run.yaml=run.yaml -n redhat-ods-operator
```

4. Create a **ogx-server.yaml** file with the following content:

```
apiVersion: ogx.io/v1beta1
kind: OGXServer
metadata:
  name: ogx-server
  namespace: redhat-ods-operator
spec:
  distribution:
    name: rh-dev
  workload:
    replicas: 1
  overrides:
    env:
      # vLLM provider configuration
      - name: VLLM_URL
        value: "https://your-vllm-service:8000/v1"
      - name: VLLM_API_TOKEN
        value: "your-vllm-token"
      - name: VLLM_TLS_VERIFY
        value: "false"
      # OpenAI provider configuration
      - name: OPENAI_API_KEY
        value: "your-openai-api-key"
      - name: OPENAI_BASE_URL
        value: "https://api.openai.com/v1"
  userConfig:
    configMapName: ogx-custom-config
    configMapNamespace: redhat-ods-operator
```

5. Apply the distribution by running the following command:

```
$ oc apply -f ogx-server.yaml
```

6. Wait for the distribution to be ready by running the following command:

```
$ oc wait --for=jsonpath='{.status.phase}'=Ready ogxserver/ogx-server -n redhat-ods-operator --timeout=300s
```

7. Generate OAuth tokens for each user account to authenticate API requests.

- a. To request a basic access token and save it to a **user1.token** file, run the following command:

```
$ curl -d client_id=ogx \
  -d client_secret=YOUR_CLIENT_SECRET \
  -d username=user1 \
  -d password=user1-password \
  -d grant_type=password \
  https://my-keycloak-server.com/realms/ogx-demo/protocol/openid-connect/token \
  | jq -r .access_token > user1.token
```

- b. To request a token for the privileged user and save it to a **user2.token** file, run the following command:

```
$ curl -d client_id=ogx \
  -d client_secret=YOUR_CLIENT_SECRET \
  -d username=user2 \
  -d password=user2-password \
  -d grant_type=password \
  https://my-keycloak-server.com/realms/ogx-demo/protocol/openid-connect/token \
  | jq -r .access_token > user2.token
```

- c. Verify the token claims by running the following command:

```
$ cat user2.token | cut -d . -f 2 | base64 -d 2>/dev/null | jq .
```

Verification

1. Set the OGX service URL:

```
$ export OGX_HOST="http://<ogx-host>:8321"
```

2. Verify basic access for **user1**, who has no privileged roles.
Load the token:

```
$ USER1_TOKEN=$(cat user1.token)
```

Confirm that **user1** can access the vLLM-served model:

```
$ curl -s -o /dev/null -w "%{http_code}\n" \
  -X POST "${OGX_HOST}/v1/openai/chat/completions" \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer ${USER1_TOKEN}" \
  -d '{"model": "vllm-inference/llama-3-2-3b", "messages": [{"role": "user", "content": "Hello!"}], "max_tokens": 50}'
```

Expected result: HTTP **200**.

Confirm that **user1** is denied access to the restricted OpenAI model:

```
$ curl -s -o /dev/null -w "%{http_code}\n" \
-X POST "${OGX_HOST}/v1/openai/chat/completions" \
-H "Content-Type: application/json" \
-H "Authorization: Bearer ${USER1_TOKEN}" \
-d '{"model":"openai/gpt-4o-mini","messages":
[{"role":"user","content":"Hello!"}], "max_tokens":50}'
```

Expected result: HTTP **403**.

3. Verify privileged access for **user2**, who is assigned the **inference_max** role.
Load the token:

```
$ USER2_TOKEN=$(cat user2.token)
```

Confirm that **user2** can access both models:

```
$ curl -s -o /dev/null -w "%{http_code}\n" \
-X POST "${OGX_HOST}/v1/openai/chat/completions" \
-H "Content-Type: application/json" \
-H "Authorization: Bearer ${USER2_TOKEN}" \
-d '{"model":"vllm-inference/llama-3-2-3b","messages":
[{"role":"user","content":"Hello!"}], "max_tokens":50}'
```

```
$ curl -s -o /dev/null -w "%{http_code}\n" \
-X POST "${OGX_HOST}/v1/openai/chat/completions" \
-H "Content-Type: application/json" \
-H "Authorization: Bearer ${USER2_TOKEN}" \
-d '{"model":"openai/gpt-4o-mini","messages":
[{"role":"user","content":"Hello!"}], "max_tokens":50}'
```

Expected result: HTTP **200** for both requests.

4. Verify that requests without a Bearer token are denied.

```
$ curl -s -o /dev/null -w "%{http_code}\n" \
-X POST "${OGX_HOST}/v1/openai/chat/completions" \
-H "Content-Type: application/json" \
-d '{"model":"vllm-inference/llama-3-2-3b","messages":
[{"role":"user","content":"Hello!"}], "max_tokens":50}'
```

Expected result: HTTP **401**.

CHAPTER 13. CONFIGURE ATTRIBUTE-BASED ACCESS CONTROL (ABAC) ON YOUR OGX SERVER

OGX supports OAuth 2.0/OIDC authentication with attribute-based access control (ABAC) for multi-tenant isolation. ABAC provides multi-tenant isolation by configuring access policies based on specific attributes, assigned to a user and the requested resource. When enabled, users can only access resources they own based on the attributes, and system resources are readable by all authenticated users.

The following procedure describes how to enable attribute-based access control (ABAC) policies in your OGX distribution.

Prerequisites

- You have installed OpenShift 4.19 or later.
- You have logged in to Red Hat OpenShift AI.
- You have cluster administrator privileges for your OpenShift cluster.
- You have access to an OAuth 2.0/OIDC identity provider, for example, a Keycloak provider.

Procedure

1. The **AUTH_*** parameters also need to be set in the **OGXServer** custom resource. For example:

Example OGXServer CR

```
spec:
  replicas: 1
  server:
    containerSpec:
      env:
      ...
      - name: AUTH_ISSUER
        value: https://keycloak-redhat-ods-applications.apps.rosa.<user-cluster>.gm8d.p3.openshiftapps.com/realms/ogx-demo
      - name: AUTH_JWKS_URI
        value: http://keycloak:8080/realms/ogx-demo/protocol/openid-connect/certs
```

The **server.auth** section of the **config.yaml** file includes the authentication environment variables, these specifications uses OAuth2 token validation:

```
server:
  auth:
    provider_config:
      type: ${env.AUTH_ISSUER:+oauth2_token}
      audience: ${env.AUTH_AUDIENCE:=ogx}
      issuer: ${env.AUTH_ISSUER:=}
    jwks:
      uri: ${env.AUTH_JWKS_URI:=}
      key_recheck_period: ${env.AUTH_JWKS_RECHECK_PERIOD:=3600}
      verify_tls: ${env.AUTH_VERIFY_TLS:=true}
```

Table 13.1. Environment variables reference

Variable	Description	Default
AUTH_ISSUER	OpenID connect (OIDC) issuer URL. If unset, authentication is disabled	None
AUTH_AUDIENCE	Expected token audience	ogx
AUTH_JWKS_URI	JSON Web key set (JWKS) endpoint for token validation	None
AUTH_JWKS_RECHECK_PERIOD	How often, in seconds, to refresh JWKS keys	3600
AUTH_VERIFY_TLS	Verify TLS when fetching JWKS	true

- The client user must include a valid JWT bearer token in requests, for example:

```
$ curl -H "Authorization: Bearer <token>" \
https://ogx.example.com/v1/models
```

- The OGX distribution ships with a default access policy:

```
access_policy:
- permit:
  actions: [read]
  when: resource is unowned
  description: "All users can read system resources"
- permit:
  actions: [create]
  description: "Authenticated users can create resources"
- permit:
  actions: [read, update, delete]
  when: user is owner
  description: "Owners can manage their own resources"
```

You can change these policies and create custom permissions for resource allocation.

The default policy describes the following behaviors for users:

- **System resources are readable by all** – Resources without an owner. Models, shields, benchmarks registered in configuration are readable by any authenticated user.
- **Any authenticated user can create resources** – Users can create their own vector databases, files, datasets, conversations, etc.
- **Users can only manage their own resources** – Read, update, and delete operations on owned resources are restricted to the resource owner.

This access policy applies to user-created resources including: Vector databases, Files, Datasets, Conversations, Responses, Agents. While system resources registered in the **config.yaml** file do not have an owner and are accessible by all user types.

CHAPTER 14. SELF-SIGNED CERTIFICATES WITH OGX

You can configure a **OGXServer** custom resource (CR) to trust certificates that are issued by self-signed or private Certificate Authorities (CAs). This configuration enables the OGX server to establish secure TLS connections to external inference, embedding, or vector store providers.

To configure a custom CA bundle, you reference a config map that contains the CA certificates from the **spec.server.tlsConfig.caBundle** field of the CR. The OGX Operator validates the certificates, mounts a concatenated bundle into the OGX server pod, and sets the **SSL_CERT_FILE** environment variable so that TLS clients in the server trust the bundle automatically.



IMPORTANT

When you configure or change the CA bundle for a **OGXServer** CR, the OGX Operator restarts the OGX server pod so that the new certificates take effect. Plan for a brief service interruption when you apply or update the CA bundle on a **OGXServer** CR that is serving production traffic.

For the procedure and the OGX Operator processing details, see [Configuring a CA bundle for OGX](#) in *Installing and uninstalling Red Hat OpenShift AI*.

CHAPTER 15. ENABLING HIGH AVAILABILITY AND AUTOSCALING FOR OGX

You can configure OGX servers to remain available if a pod restarts, an application crashes, or node maintenance occurs. You can also enable autoscaling to adjust server capacity automatically based on resource usage. This procedure shows how to configure high availability and autoscaling for OGX server pods by using the **OGXServer** custom resource.

Prerequisites

- You have installed OpenShift 4.19 or later.
- You have logged in to Red Hat OpenShift AI.
- You have cluster administrator privileges for your OpenShift cluster.
- You have activated the OGX Operator in OpenShift AI. For more information, see [Activating the OGX Operator](#).
- You have installed the OpenShift CLI (**oc**) as described in the documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS

Procedure

1. To enable high availability for your OGX server, add the following parameters to your **OGXServer** CR:

```
spec:
  replicas: 2 ❶
  server:
    podDisruptionBudget:
      maxUnavailable: 1 ❷
    topologySpreadConstraints: ❸
      - maxSkew: 1 ❹
        topologyKey: topology.kubernetes.io/zone ❺
        whenUnsatisfiable: ScheduleAnyway ❻
      labelSelector:
        matchLabels:
          app.kubernetes.io/instance: ogxserver-sample ❼
```

- ❶ Runs two OGX pods for high availability.
- ❷ Specifies voluntary disruption tolerance. This configuration keeps at least one server pod available during voluntary disruptions.
- ❸ Specifies how matching pods are spread across the cluster topology.
- ❹ Instructs the scheduler to minimize replica imbalance across zones. With two replicas, the scheduler attempts to place one pod per zone.
- ❺ Uses the node zone label as the failure domain for pod spreading.

- 6 Allows scheduling to proceed even if spread constraints cannot be fully satisfied.
- 7 Ensures that only pods from the same application instance are considered when calculating spread.

2. To enable autoscaling for your OGX server, add the following parameters to your **OGXServer** CR:

```
spec:
  server:
    autoscaling: 1
    minReplicas: 1 2
    maxReplicas: 5 3
    targetCPUUtilizationPercentage: 75 4
    targetMemoryUtilizationPercentage: 70 5
```

- 1 Configures a HorizontalPodAutoscaler (HPA) for the server pods.
- 2 Specifies the minimum number of replicas maintained by the HPA.
- 3 Specifies the maximum number of replicas maintained by the HPA.
- 4 Enables CPU-based scaling.
- 5 Enables memory-based scaling.

Additional information

- [Controlling pod placement by using pod topology spread constraints](#)
- [Automatically scaling pods with horizontal pod autoscaling](#)